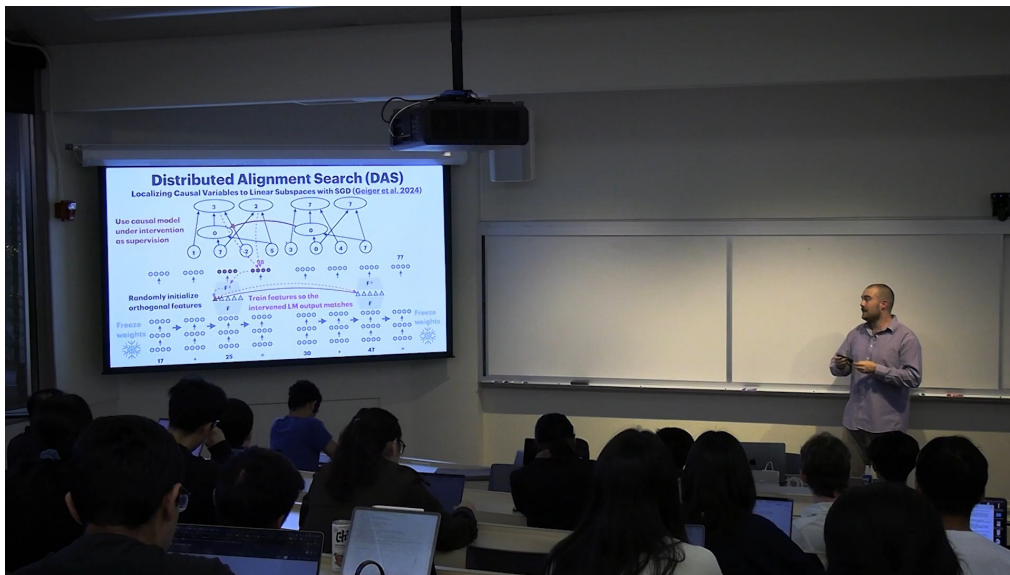


课程笔记

因果机制可解释性：从激活转向到反事实机制定位

Stanford 课程 Lecture 1

整理：Codex



视频标题：因果机制可解释性 Lecture1 | Stanford 课程

视频来源：Bilibili BV1MmQwBSEWT

发布者：至高机器智能

视频时长：01:15:11

字幕来源：Bilibili 英文字幕，整理阶段使用英文，正文写作为中文。

目录

1 因果视角下的机制可解释性	2
1.1 为什么要给“机制”下定义	2
1.2 从文化标签到技术定义	3
1.3 本章小结	3
2 激活转向：用向量干预改变模型行为	4
2.1 Golden Gate Claude 带来的直观入口	4
2.2 差值均值转向：从对比数据到转向向量	5
2.3 AxBench：控制方法如何被评估	6
2.4 转向方法的边界：控制效果与研究价值	7
2.5 本章小结	8
3 因果中介分析：从输入扰动到内部路径	8
3.1 中介变量：总效应怎样穿过第三个变量	8
3.2 神经网络版本：输入、内部组件与输出	9
3.3 Causal Tracing：Space Needle 到 Seattle 的事实检索	10
3.4 MLP、注意力头与残差流的分工	12
3.5 从噪声消融到反事实补丁	12
3.6 本章小结	13
4 因果抽象：算法如何嵌入复杂系统	13
4.1 什么算作计算性解释	13
4.2 把神经网络和算法都写成因因果模型	14
4.3 两位数加法中的 carry-the-one 变量	15
4.4 结构对应：同一干预在两个层级上的同构效果	16
4.5 抽象层级与解释深度	17
4.6 本章小结	18
5 变量定位：从线性表征到可训练干预	18
5.1 为什么反事实补丁更贴近模型自然分布	18
5.2 线性表征假说：变量可能藏在任意方向	19
5.3 分布式交换干预	20
5.4 用因果模型监督特征方向	21
5.5 基于 SAE 的掩码选择	23
5.6 和线性探针的关键区别	23
5.7 本章小结	24

6 反事实数据集中的 look-back 机制	26
6.1 从单个二元变量走向复杂任务	26
6.2 Bob、Carla、杯子与咖啡：模板任务的结构	26
6.3 三阶段机制：标记、查询、取回答案	27
6.4 为什么要构造精确反事实	28
6.5 位置指针与最终 token 的分离证据	29
6.6 子空间大小、相对位置与 tokenization 限制	30
6.7 本章小结	32
7 总结与延伸：拥抱混合机制	32
7.1 热图为何看起来平滑或尖锐	32
7.2 两项列表中的清晰机制	33
7.3 长列表中的 U 形退化与词汇直查	33
7.4 从清晰个案到混合机制的研究姿态	33
7.5 本章小结	34
7.6 全课方法速查	34

1 因果视角下的机制可解释性

1.1 为什么要给“机制”下定义

课程一开始，讲者把机制可解释性放在一个仍处于“范式形成期”的研究现场中：大家已经在打开黑箱、画特征、做补丁、找回路，但“机制”这个词本身还带着多层含义。00:00:07–00:00:22 的开场给出了本讲的主线：用因果性来理解黑箱系统。换句话说，读者接下来看到的每一种方法都要回答同一个问题：如果研究者干预模型内部的某个对象，模型行为会不会按预测改变？

这里的“机制可解释性”指 mechanistic interpretability。按讲者采用的窄技术定义，它研究神经网络内部如何通过因果机制产生行为。这个定义比“看模型里面有什么”更尖锐，因为它要求解释经得起干预检验：一个看起来有意义的神经元、方向、注意力头或残差流片段，只有在被替换、添加、移除、固定或反事实交换时能稳定改变输出，才进入“机制”的讨论范围。

本讲采用的工作定义

机制可解释性，是通过可干预、可验证的因果机制来理解神经网络内部计算的研究。一个因果机制至少包含三层信息：内部变量是什么，它怎样参与计算，干预它以后输出为什么会按某个方向改变。

这个定义背后有一个很实用的判断标准。普通的相关性观察像是在房间外听机器声音：声音变大时产量也许提高了，但很难知道哪根轴在真正传力。因果机制更像打开机器后拨动某个齿轮：如果拨动齿轮 Z 会让输出 Y 按预期改变，研究者才有理由说 Z 参与了产生 Y 的过程。神经网络没有可见齿轮，只有向量、层、头、MLP 模块、残差流和特征方向等后文会逐步定义的对象；机制可解释性的任务就是把这些对象变成可检验的计算部件。

¹视频时间：00:00:20–00:01:25。来源帧：00:01:10，截图来自课程视频。

Mechanistic?

Saphra and Wiegrefe (2024)

Narrow Cultural Definition

AI safety researchers motivated by philosophical arguments for interpretability.

Broad Cultural Definition

AI researchers interested in model internals.

Broad Technical Definition

Any research that describes the internals of a model.

Narrow Technical Definition

Understanding neural networks through their causal mechanisms.

图 1: 开场的“Mechanistic?”定义框架：讲者把“机制”拆成文化定义和技术定义，并突出狭义技术定义，即通过因果机制理解神经网络内部。这个定义给后续的激活转向、中介分析和因果抽象建立了共同坐标。¹

1.2 从文化标签到技术定义

00:00:24–00:01:25 这一段引用了题为“Mechanistic?”的观点文章，用四种含义拆开“mechanistic”这个词。第一种是窄文化含义：一批 AI safety 研究者因为世界风险、对齐问题等哲学和安全动机投入该领域。第二种是宽文化含义：研究者普遍想窥视黑箱，弄清模型为什么能做出看似聪明的事。第三种是宽技术含义：只要研究模型内部结构和表征，都可以归入机制可解释性。第四种是讲者偏好的窄技术含义：通过因果机制理解神经网络内部。

“机制”一词的四个层级

同一个词在研究社区里可能同时承担身份标签、研究兴趣、方法集合和技术定义四种角色。本文后续采用第四种用法：机制是一组能被干预验证的内部计算关系。读文献时如果不先区分这四层，很容易把“研究者属于哪个圈子”和“某个解释是否经得起因果检验”混在一起。

这也解释了本讲的章节推进。00:01:32–00:01:51 中，讲者预告会先从激活转向开始，因为它直观展示“改变内部向量可以改变行为”；随后进入因果中介分析，因为它追问“影响从输入到输出经过哪些内部路径”；再进入因果抽象，因为它要检验“网络内部是否实现了一个明确算法”；最后讨论反事实输入，因为复杂任务里只有精确构造对样本，研究者才能分开相互缠绕的机制。

常见误解：看到内部结构不等于找到因果机制

把注意力图、神经元激活或 SAE 特征标签展示出来，只能说明研究者观察到了某种内部模式。要把它提升为机制解释，还需要展示干预效果：改变这个内部对象时，目标输出、行为倾向或中间变量会按解释所预测的方式变化。

1.3 本章小结

本章的作用是定标。机制可解释性在本讲中采用窄技术定义：研究神经网络内部的因果机制。因果机制强调“可干预的内部变量”和“可预测的行为变化”之间的关系。后续章节里的激活转向、因果中介分析、因果抽象和反事实数据集设计，都可以看成同一个问题的不同推进：研究者

怎样从内部向量的可控变化，走向对模型计算过程的结构性解释。

2 激活转向：用向量干预改变模型行为

2.1 Golden Gate Claude 带来的直观入口

00:01:54–00:02:35 的 Golden Gate Claude 是激活转向的直观入口。Anthropic 曾发布一个特殊版本的 Claude：研究者把一个被标记为 Golden Gate Bridge 的内部特征加入模型内部表征，模型随后频繁以金门大桥的身份说话，或不断把话题拉回金门大桥。这个案例之所以有冲击力，并不在于金门大桥本身多特殊，而在于它把一个抽象事实变得可感知：模型内部向量空间中的某个方向，经过干预后会对下游语言行为产生可理解的影响。

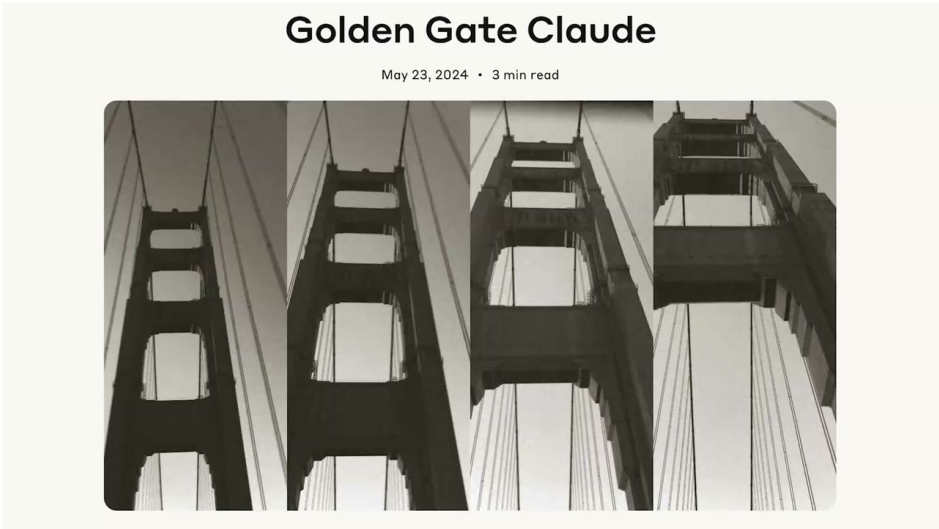


图 2: Golden Gate Claude 作为激活转向的入口案例：向模型内部加入与 Golden Gate Bridge 相关的特征后，模型行为会稳定地向这个主题偏移。它展示的关键点是“内部表示可被干预，干预会改变外部行为”。²

激活转向（activation steering）就是沿着这个思路发展出来的方法族。最简洁的说法是：研究者在模型运行时向某一层或某个内部位置加入一个向量，以改变模型接下来的输出行为。被加入的向量称为转向向量（steering vector）。它可以来自人工构造、监督训练，也可以来自一组对比样本的激活差异。

激活转向的核心机制

激活转向把模型内部状态当作可干预对象：研究者在选定层或位置加入一个行为方向，再观察下游输出是否按预期改变。这里的核心在于把相关方向推进为一次可复现的内部干预，并检查行为变化是否跟随该方向。

这里的“干预”可以先按朴素方式理解：让一个变量取它自然运行时本来不会取的值。比如原始提示词 x 可能让模型自然产生一条关于婚礼的正面回答；研究者在中间层加入“讨厌婚礼”的转向向量后，输出变成负面回答。这个操作已经带有因果味道，因为它改变了内部状态，并观察到了后续行为变化。

²视频时间：00:01:54–00:02:51。来源帧：00:02:10，截图来自课程视频。

2.2 差值均值转向：从对比数据到转向向量

00:03:00–00:04:14 讲者介绍了 difference-in-means steering，即差值均值转向。例子故意很简单：假设模型对“你妹妹的婚礼怎么样？”回答得过于积极，研究者想把它转向“讨厌婚礼”的行为。做法是收集两组提示词：一组表达喜欢婚礼，一组表达讨厌婚礼；把它们输入同一个语言模型；在选定层收集内部激活；分别求两组激活的均值；再用均值之差得到一个方向。

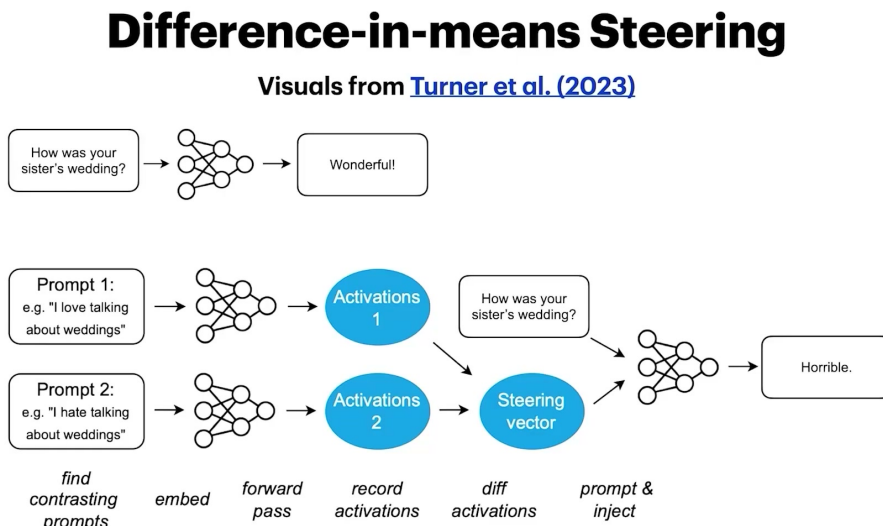


图 3: 差值均值转向的完整流程：收集“喜欢婚礼”和“讨厌婚礼”的对比提示，前向运行模型并记录激活，对两组激活分别取均值，再用均值差得到转向向量，最后把该向量注入到普通提示的模型运行中以改变回答方向。³

公式要表达的事情很直接：用两类样本在同一内部位置产生的平均激活差，近似表示“从一种行为倾向移动到另一种行为倾向”的方向。若目标是把模型推向负类行为，例如“讨厌婚礼”，可以采用下面的符号约定：

$$\bar{a}_{+, \ell} = \frac{1}{|D_+|} \sum_{x \in D_+} a_\ell(x), \quad \bar{a}_{-, \ell} = \frac{1}{|D_-|} \sum_{x \in D_-} a_\ell(x), \quad v_{\text{steer}} = \bar{a}_{-, \ell} - \bar{a}_{+, \ell}.$$

符号说明：

- D_+ ：正类对比提示集合，在婚礼例子中可以是“喜欢婚礼”的提示词集合。
- D_- ：负类对比提示集合，在婚礼例子中可以是“讨厌婚礼”的提示词集合。
- $a_\ell(x)$ ：输入 x 经过模型时，在第 ℓ 层或指定内部位置产生的激活向量。
- $\bar{a}_{+, \ell}$ ：正类提示在该内部位置上的平均激活。
- $\bar{a}_{-, \ell}$ ：负类提示在该内部位置上的平均激活。
- v_{steer} ：从正类均值指向负类均值的转向向量；若想转向正类，可以反过来取 $\bar{a}_{+, \ell} - \bar{a}_{-, \ell}$ 。

得到 v_{steer} 后，运行新提示 x_{test} 时可以在同一层执行：

³视频时间：00:03:00–00:04:18。来源帧：00:04:00，截图来自课程视频。

$$a'_\ell(x_{\text{test}}) = a_\ell(x_{\text{test}}) + \alpha v_{\text{steer}}.$$

符号说明：

- x_{test} ：实际要控制的新提示词。
- $a'_\ell(x_{\text{test}})$ ：加入转向向量后的内部激活。
- α ：转向强度系数；数值越大，干预越强，也越可能破坏流畅性或提示遵循。

符号方向要说清楚

差值均值转向没有天然固定的正负号。若 $v_{\text{steer}} = \bar{a}_- - \bar{a}_+$ ，它把模型推向负类；若改成 $\bar{a}_+ - \bar{a}_-$ ，方向就反过来。写实验报告时必须同时说明对比集合、采样层、注入位置和符号约定，否则同一个公式可能对应相反行为。

差值均值转向的吸引力来自低门槛：它不要求重新训练整个模型，只需要对比数据和内部激活访问权限。它的局限也在这里：均值差只能捕捉粗粒度方向，样本选择、层位置、注入强度和行为目标都会影响结果。对于“讨厌婚礼”“谈论金门大桥”这类粗目标，它容易展示效果；对于“用法语、幽默地、准确地谈论金门大桥，同时保持原始问题约束”这类多属性目标，它会迅速变得脆弱。

2.3 AxBench：控制方法如何被评估

00:04:18–00:05:18 中，讲者把激活转向放进更宽的“控制方法”框架。AxBench 的设置是：给定一个用户提示和一个转向提示，某种控制方法输出一个被改变后的模型或运行过程；评估者再看生成文本是否同时满足三个条件。第一，连贯性：输出仍然是正常语言。第二，提示遵循：模型仍然回答原始用户问题。第三，转向效果：输出确实体现了目标行为，例如以 Golden Gate Bridge 的身份回答。

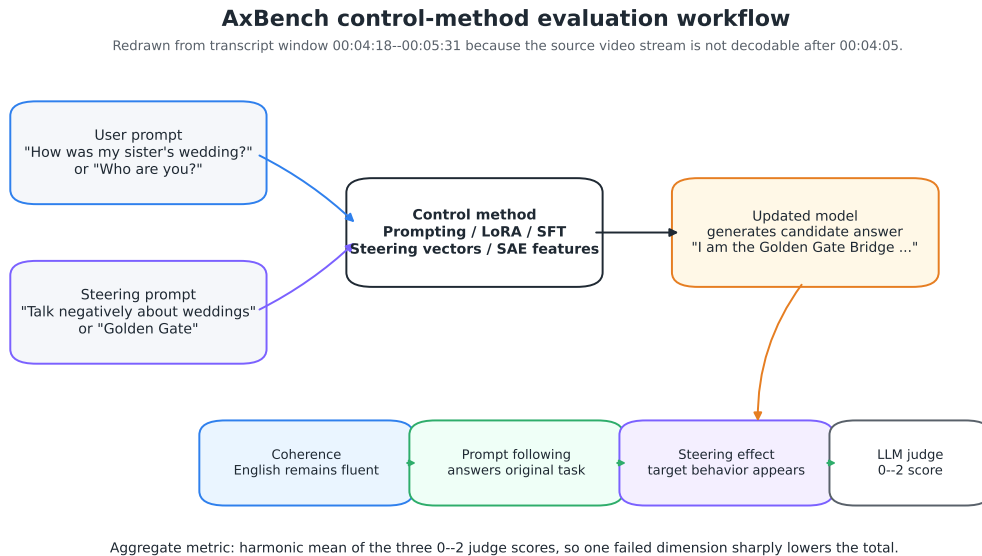


图 4: AxBench 的控制评估设置：输入普通用户提示和转向提示，控制方法产生一个“更新后的模型”或等效受控行为，再由 LLM 裁判分别评估连贯性、遵循原提示和转向效果。这里使用重绘图，因为对应视频区间的源视频流无法解码出可读截图。⁴

00:05:19–00:05:49 进一步说明了评分机制：每个维度由 LLM 裁判分别给 0–2 分，然后取调和平均。调和平均是一个很苛刻的聚合方式，因为任一维度为 0，整体分数就会变成 0。这迫使控制方法同时做好三件事：不能胡言乱语，不能丢掉用户问题，也不能忘记转向目标。

为什么用调和平均

调和平均会惩罚短板。算术平均允许一个维度很高、另一个维度很低时仍得到中等分；调和平均更像三脚架，只要一条腿断掉，整体就站不稳。对模型控制来说，这很合理：一个只会谈金门大桥却不回答问题的模型，不能算成功控制。

00:05:55–00:07:58 的结果给出一个现实判断：如果目标只是让模型表现出某种行为，提示词和微调通常更强。用户直接要求模型“假装成金门大桥”，模型往往就会照做；LoRA 或监督微调也能把目标行为写进模型参数。激活转向在这个控制竞赛里弱于最强工程方案；它的研究价值在于提供一个轻量入口：不用重训整个网络，就能测试内部方向与行为之间的因果联系。

AxBench qualitative results: control strength vs. research value		
Redrawn from transcript window 00:05:31–00:07:54; no numeric values are invented without a readable source table.		
Method family	Relative result stated in lecture	What the result means
Prompting LoRA / SFT	Strongest practical control methods	Best baseline when the goal is simply to produce a model with the desired behavior.
Supervised steering vector	Best among steering-vector methods	The injected vector is optimized directly for the desired behavior.
Difference-in-means steering	Decent steering; weaker than supervised	Contrast prompts -> average activations -> subtract means -> inject direction.
SAE labeled features	Weak on average for control	A labeled SAE feature is not automatically a high-leverage control knob.

Metric: coherence, prompt following, and steering effect are each judged by an LLM on 0–2 scores, then combined by harmonic mean. Lecture takeaway: practical control favors prompting/fine-tuning; internal-intervention research favors supervised steering and difference-in-means over direct SAE-label steering.

图 5: AxBench 结果的讲者版解读：提示和微调是最强的实用控制基线；若研究目标限定为内部转向，监督训练的转向向量与差值均值转向强于直接使用 SAE 标签特征。评估指标使用三项 LLM 裁判分数的调和平均，因此任一维度失败都会显著压低总分。⁵

2.4 转向方法的边界：控制效果与研究价值

讲者在 00:07:01–00:07:48 区分了几类转向向量。监督转向向量可以直接针对目标行为优化，通常强于差值均值方法。差值均值转向不用目标优化，只靠对比激活，效果较弱但更简单。直接使用带标签的稀疏自编码器特征进行转向，在该基准上平均表现并不理想；这提醒读者，自动解释标签听起来可读，实际控制能力还要用干预实验检验。

00:08:27–00:10:24 的问答补上了一个重要边界：当前转向目标多半粗粒度。历史上有研究把差值均值方法用于 truthfulness steering，即朝“更真实”的回答方向转向；但若把目标拆成多个同时约束的属性，比如既要谈金门大桥，又要用法语，还要幽默并保持事实准确，现有转向方法就很难稳定处理。AxBench 的任务也因此偏向可评估、可对照的粗目标。

⁴视频时间：00:04:18–00:05:31。图为根据字幕重绘的教学示意图，未伪造成原始截图。
⁵视频时间：00:05:31–00:07:54。图为根据字幕重绘的定性结果示意图；因无可读表格截图，未编造具体数值。

不要把“可转向”误读成“已理解机制”

激活转向能说明某个内部方向对行为有因果影响，但它通常还没有说明这个方向在模型自然完成任务时怎样被产生、怎样被读取、怎样与其他部件协作。它回答了“我能不能推模型一下”，后续的因果中介分析和因果抽象才会继续追问“模型原本怎样自己完成这件事”。

本节的研究定位

从工程控制角度看，提示词和微调往往更有效；从机制研究角度看，激活转向提供了最直观的干预范式。它证明内部向量可以被当作因果把手使用，也暴露了下一步问题：研究者需要从“改变行为的方向”走向“产生行为的内部路径”。

2.5 本章小结

激活转向把机制可解释性的因果直觉落到了一个可操作动作上：在模型内部加入转向向量，观察输出是否按目标行为变化。Golden Gate Claude 展示了内部特征干预的直观效果；差值均值转向展示了如何从对比样本的平均激活中构造方向；AxBench 则把控制方法放到连贯性、提示遵循和转向效果三个维度上评估。它的关键启发很清楚：转向是理解内部因果影响的入口，但要解释模型自然计算过程，还需要追踪输入、内部组件和输出之间的因果路径。

3 因果中介分析：从输入扰动到内部路径

3.1 中介变量：总效应怎样穿过第三个变量

激活转向展示了一个直接事实：只要能改写模型内部激活，就能改变模型行为。因果中介分析追问更细的问题：当输入 X 改变输出 Y 时，这个影响究竟经过了哪个内部变量 Z ？在普通因果图里，最小图景可以画成 $X \rightarrow Z \rightarrow Y$ ，同时允许一条绕过 Z 的路径 $X \rightarrow Y$ 。这时 Z 称为中介变量，也就是把上游原因的一部分影响传给下游结果的第三个变量。

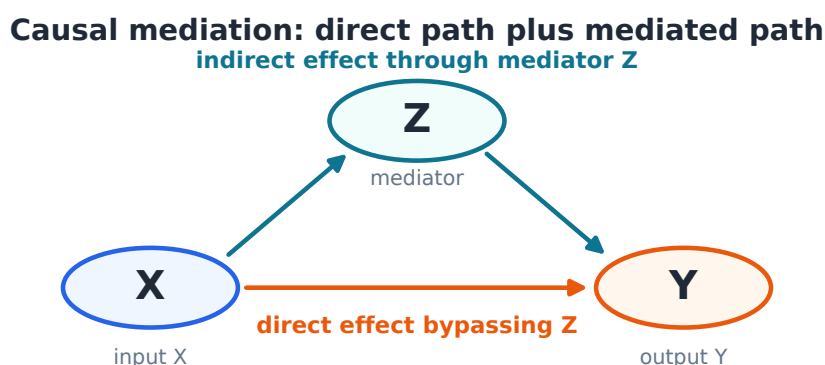


图 6: 因果中介分析的最小 DAG：输入 X 可以经由中介变量 Z 影响输出 Y ，也可以沿 $X \rightarrow Y$ 的直接路径影响输出。这个图对应总效应、直接效应和间接效应的三分法。⁶

⁶源视频时间：00:10:24-00:11:25。本图根据该片段讲解重绘；本地源视频在该时间窗无法稳定解码出可读截图。

中介分析的三个效应

总效应关心“把输入从 x_0 改成 x_1 ，输出整体变了多少”；直接效应关心“中介变量保持在原来的状态时，输入本身还能带来多少变化”；间接效应关心“输入提供的中介状态变化，能单独带来多少输出变化”。

为了让这三个量精确一些，可以把输出写成潜在输出 $Y_{x,z}$ ：它表示把输入固定为 x ，同时把中介变量固定为 z 时，系统会产生的输出分数、概率或 logit。令 $Z(x)$ 表示在输入为 x 时，中介变量自然会取到的值。于是一个常用的教学版本可以写成：

$$TE = Y_{x_1, Z(x_1)} - Y_{x_0, Z(x_0)}$$

其中，TE 是总效应； x_0 是基准输入； x_1 是被干预后的输入； $Z(x_0)$ 和 $Z(x_1)$ 分别是中介变量在两个输入下自然出现的值； $Y_{x, Z(x)}$ 是系统在该输入和该中介状态下的输出量。

$$NDE = Y_{x_1, Z(x_0)} - Y_{x_0, Z(x_0)}$$

其中，NDE 是自然直接效应；两个项里的中介变量都被固定在 $Z(x_0)$ ，所以比较的是输入从 x_0 换成 x_1 后，绕开中介变化还能产生的输出差异。

$$NIE = Y_{x_1, Z(x_1)} - Y_{x_1, Z(x_0)}$$

其中，NIE 是自然间接效应；两个项里的输入都固定为 x_1 ，所以比较的是中介变量从 $Z(x_0)$ 换成 $Z(x_1)$ 后，经过中介路径带来的输出差异。在线性、可加的直觉图景里，总效应可以被理解为直接效应和间接效应的组合；在神经网络里，这种分解更像一组反事实实验的语言，帮助研究者说清楚“哪个内部状态承担了多少因果责任”。

3.2 神经网络版本：输入、内部组件与输出

把这套语言搬进语言模型以后， X 通常是提示词输入， Y 是模型输出， Z 则可以是一颗神经元、一个激活向量、一个注意力头、一个 MLP 模块，或者某一层某个 token 位置上的残差流切片。这里的“中介”没有神秘含义，它只是可被读出、替换、固定的内部状态；如果替换它会系统性改变输出，它就有资格被当作候选中介变量。

讲者给出的例子是把提示词中的“nurse”改成“man”。这个输入编辑会改变模型输出，这是总效应。现在研究者可以重新运行模型，并把某个神经元 Z 固定为它在“nurse”输入下原本会取到的值。这样得到的输出变化，就是在“输入已经换成 man，但中介仍保持 nurse 状态”的条件下测得的直接效应。反过来，也可以保留“nurse”输入，却把 Z 固定为它在“man”输入下会取到的值；这对应经过 Z 的间接效应。

为什么这个例子重要

普通输入编辑只能告诉研究者“输出变了”。中介干预进一步把问题拆成两层：输入文本本身造成了什么变化，内部组件 Z 携带的状态变化又造成了什么变化。对机制可解释性来说，第二层才是通往“路径解释”的入口。

这里的关键动作是固定、替换或恢复激活。这个动作后来会以多个名字出现：神经元钳制、概念擦除、激活补丁（activation patching，也就是激活替换）、组件恢复。名称不同，核心都在做同

⁷源视频时间：00:11:25–00:12:59。本图根据该片段讲解重绘。

Mediator intervention on an LM prompt edit

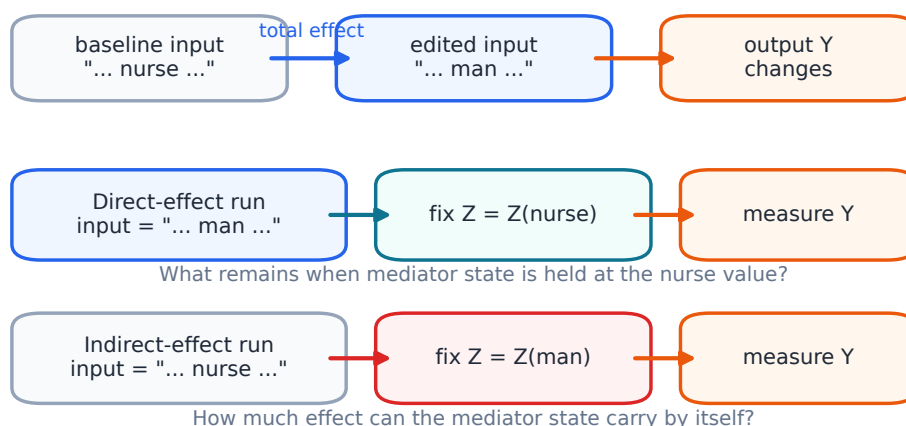


图 7：“nurse”与“man”的中介干预实验：直接效应运行把输入换成“man”，但把 Z 固定为“nurse”输入下的值；间接效应运行保留“nurse”输入，却把 Z 固定为“man”输入下的值。⁷

一类反事实实验：让系统某一部分取一个它在另一种输入下会自然取到的值，然后观察输出是否跟着改变。

3.3 Causal Tracing: Space Needle 到 Seattle 的事实检索

因果追踪 (causal tracing) 把中介分析落到 transformer 的事实检索任务上。输入句子是“The Space Needle is in downtown _____”，如果模型掌握了相关世界知识，目标输出应当是“Seattle”。实验先在 Space Needle 相关 token 的嵌入中加入高斯噪声。高斯噪声可以理解为一团随机扰动，它会把原本清晰的地点线索弄模糊；噪声越强，模型预测 Seattle 的概率通常越低。

接下来才是因果追踪的核心：在已经被噪声污染的运行中，研究者选择某些内部组件，把它们恢复成未污染运行时的原始激活值。如果恢复某个位置、某一层或某个组件之后，Seattle 的概率明显回升，那么这个组件就提供了一个中介证据：它携带的干净信息足以帮助模型找回原本答案。

因果追踪的实验逻辑

先破坏输入线索，让模型失去答案；再逐块恢复内部激活，看哪一块能把答案救回来。热图中的亮格子表示：恢复该层、该 token 或该组件后，目标答案 Seattle 被上调得更多。

恢复热图的横轴通常是 transformer 层，纵轴通常是输入 token 位置，格子的数值可以理解为对目标输出的恢复强度，例如 Seattle 的 logit 或概率增加量。这样，研究者就在一张图上看到事实信息怎样从 Space Needle 相关 token 出发，沿着层和 token 位置逐步移动到最终输出位置。

⁸源视频时间：00:12:59–00:14:20。本图根据该片段讲解重绘。

⁹源视频时间：00:14:20–00:16:05。本图根据该片段讲解重绘的定性示意图，未编造原文或视频中的具体数值。

¹⁰源视频时间：00:18:20–00:20:01。本图根据问答片段中的“plus or minus five layers”说明重绘；该段未获得可用视频截图。

Causal tracing: corrupt embeddings, then restore internal activations

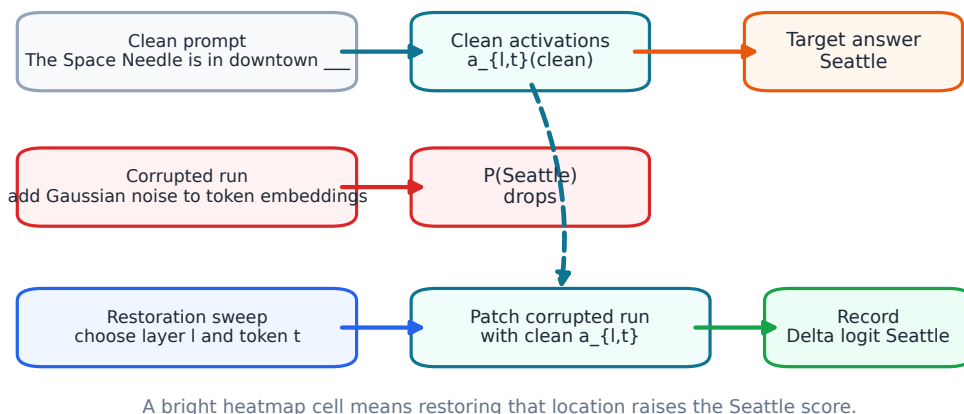


图 8: Causal tracing 的污染与恢复流程：先对 Space Needle 相关 token embedding 加入高斯噪声，使 Seattle 概率下降；再在腐化运行中逐层、逐 token 恢复干净激活，记录目标答案 Seattle 的 logit 恢复幅度。⁸

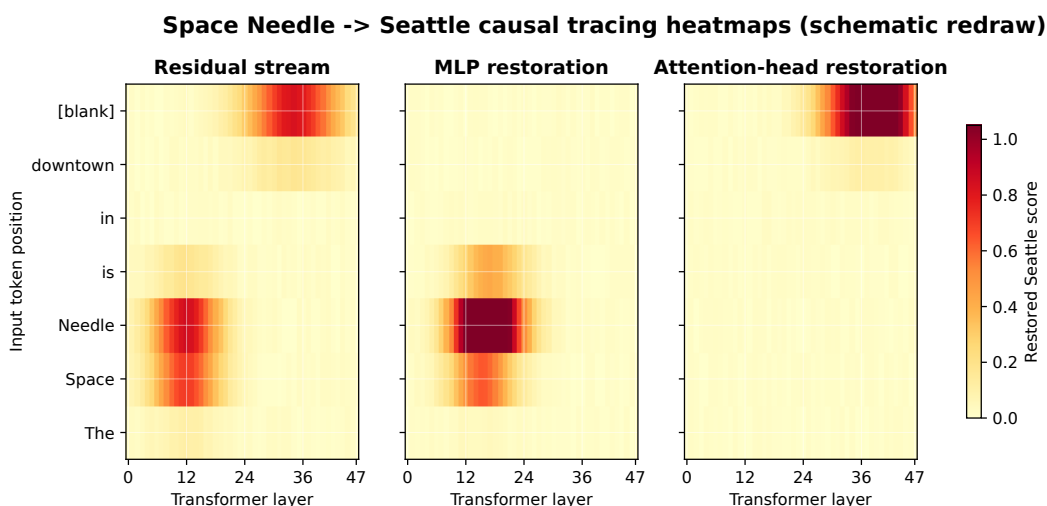


图 9: Space Needle → Seattle 的三类恢复热图示意：残差流热图展示信息从早期实体 token 迁移到最终输出位置；MLP 恢复集中在 Space/Needle 附近的较早层；注意力头恢复集中在最终 token 的后段层，用来把已取回的信息送到输出位置。⁹

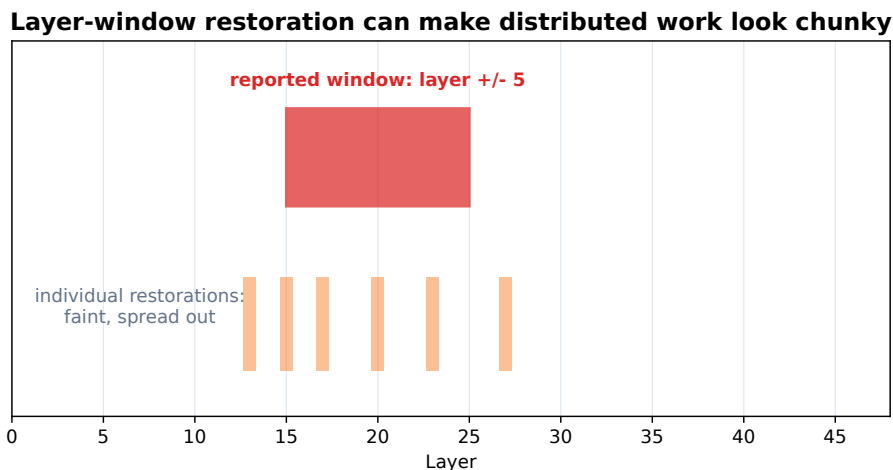


图 10: 层窗口恢复的定位 caveat: 如果一次恢复目标层前后约五层, 热图会呈现更厚的亮块; 单层或单组件恢复通常更淡、更分散, 因此亮块不应被读成唯一组件的精确定位。¹⁰

3.4 MLP、注意力头与残差流的分工

在这段讲解里, 残差流 (residual stream) 被描述为 transformer 每层之后共享的全局向量状态。它像一条贯穿网络的工作台: 每个层块都可以从里面读取信息, 也可以把处理后的信息写回去。研究者先在残差流上做恢复实验, 看到某些早期 token 和后期输出位置的恢复会让 Seattle 回来, 这给出了信息流动的粗略路线。

进一步拆开组件后, 讲者强调了一个例子级别的分工。MLP 模块似乎在较早层读取 Space Needle 相关信息, 并写出 Seattle 这个事实内容。注意力头的角色更像路由器: 它们在网络后段、最终 token 附近, 把已经计算出的 Seattle 信息搬运到最后的残差流位置, 使输出头能够给 Seattle 更高分数。

不要把例子误读成普遍定律

这段话描述的是 Space Needle 到 Seattle 这一类事实检索实验里的观察: MLP 看起来承担事实内容的取回, 注意力头看起来承担信息搬运。它不等于所有事实都只存于 MLP。讲者在问答里补充, token embedding 本身也可能存有密集的世界知识; 注意力参数更擅长在上下文中路由信息。

这个分工可以用一个简单类比理解: MLP 像查资料的柜台, 把 “Space Needle 对应 Seattle” 这条内容取出来; 注意力头像传送带, 把这条内容送到模型即将预测下一个 token 的位置; 残差流则像整条流水线上的共享托盘, 内容被不断读取、写入和移动。

3.5 从噪声消融到反事实补丁

问答部分把方法边界讲得很清楚。给 token embedding 加高斯噪声, 本质上是一种消融: 它破坏信息, 模型行为随之退化。消融能告诉研究者 “这里有用”, 但它给出的反事实比较粗, 因为污染后的输入更接近乱码状态。讲者更感兴趣的是系统性反事实: 如果把 Space Needle 换成 Colosseum, 模型应当回答 Rome; 如果换成 Eiffel Tower, 模型应当回答 Paris。这样的干预保留了任务结构, 也给研究者一个明确的期望输出。

这正是激活补丁的价值。激活补丁把某个干净输入运行中的内部激活取出来, 替换到另一个输入运行的对应位置, 再观察输出是否按预期改变。它把 “内部组件是否重要” 推进到 “内部组件是否携带某个具体反事实变量”。在后续章节里, 交换干预和因果抽象会沿着这条线继续发展。

相关术语的同一核心

概念擦除、神经元钳制、信息破坏、激活恢复、激活补丁，表面上服务于不同论文传统；从中介分析看，它们都在改写内部变量 Z ，再测量 Y 的反事实变化。差别主要在于改写方式：噪声消融让信息退化，反事实补丁把另一个真实运行中的信息搬进来。

最后还有一个很容易忽略的热图细节。因果追踪论文里的漂亮热图通常没有只恢复单独一层。为了得到更厚实、可读的图，实验会恢复目标层附近的一个窗口，例如该层前后约五层，合起来大约十个 MLP 或十组注意力头块。因此，一个亮格子可能代表一段层窗口的联合效果。讲者指出，如果查看更细的单层图，信号会更淡、更分散；这说明事实检索可能由多个 MLP 在多层中渐进完成，也可能存在更低秩、更局部的子空间参与其中。

热图的离散亮块可能夸大了定位精度

看到某个层区间很亮，不能直接推出“唯一的一个 MLP 存储了 Seattle”。窗口恢复会把多个层的贡献合并显示；单组件效果往往更分布式、更微弱。热图适合提供候选路径，精确定位还需要更细粒度的补丁、子空间分析或后续编辑实验。

3.6 本章小结

本节把因果中介分析从普通因果图推进到神经网络内部。中介变量 Z 是连接输入 X 和输出 Y 的候选内部路径；总效应、直接效应和间接效应提供了描述这条路径的反事实语言。放到 transformer 里， Z 可以是残差流切片、MLP 模块、注意力头或更小的向量子空间。

Space Needle 到 Seattle 的因果追踪展示了一个具体流程：先用高斯噪声破坏输入线索，再恢复内部组件，最后用目标答案的恢复强度画出热图。这个方法给出事实检索路径的证据，也暴露了定位上的限制：亮区常常来自层窗口恢复，单个组件的贡献可能分散在多个层和低秩方向里。后续更严格的机制解释，需要把这种“恢复答案”的证据推进到更结构化的反事实补丁和因果抽象。

4 因果抽象：算法如何嵌入复杂系统

4.1 什么算作计算性解释

这一节把视角从单个组件的因果效应拉远一步：如果一个复杂系统表现出某种行为，研究者什么时候可以说“它在执行某个算法”？这类说法称为**计算性解释**。它的对象不只包括神经网络。讲者先用笔记本电脑举例：如果外星人发现一台正在运行冒泡排序程序的电脑，好的科学解释应当恢复出某种层级结构，例如硬件状态、机器码、解释器、Python 程序，以及冒泡排序里的比较和交换。解释的关键任务是说明这些底层动态如何稳定地产生高层算法行为，无需逐个列出电路中每个电子的轨迹。

同一个思路也出现在认知科学中。认知科学长期把心智看成计算系统：研究者提出一个算法假说，再寻找大脑或行为证据，判断该算法是否真的被实现。机制可解释性面对的是另一类可逆向工程的系统，即神经网络。问题变成：神经网络的向量、层、注意力头、MLP 和残差流是否以某种方式实现了一个可写出来的算法？

计算性解释的类比

把神经网络解释成算法，类似于把一台电脑解释成程序。底层系统可以极其复杂，高层算法可以相对简洁；关键是二者之间要有可检验的对应关系。只说“这个模型像在做小学加法”还不够，研究者需要指出哪些变量、哪些干预、哪些输出变化支撑这个说法。

讲者强调，这个问题在熟悉的工程场景中显得自然：电脑运行 Python 程序时，“实现”关系几乎是定义性的。困难出现在一般黑箱系统上。若有人写下一个程序，并声称某个复杂动力系统正在执行这个程序，那么“执行”到底是什么意思？因果抽象给出的回答是：把高层算法和低层系统都写成因果模型，再比较它们在干预下的行为。

Causal abstraction: algorithm and network as causal models

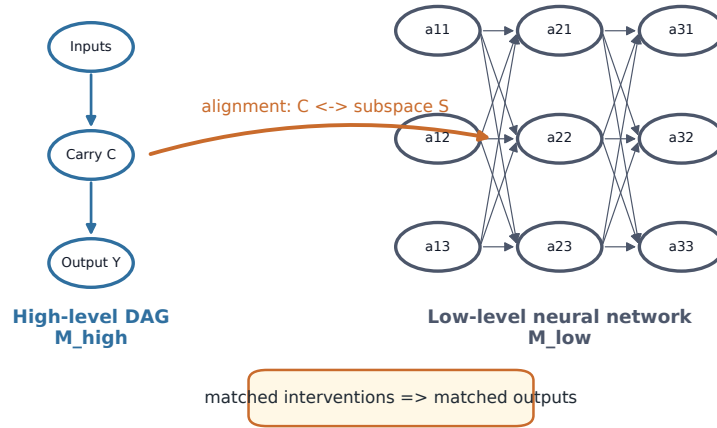


图 11: 因果抽象的统一视角：高层算法写成 M_{high} ，低层神经网络写成 M_{low} ，解释命题落在变量对齐和干预输出是否一致上。¹¹

4.2 把神经网络和算法都写成果模型

因果抽象的第一步是统一语言。神经网络看起来是连续向量计算，算法看起来是离散变量和控制流，但二者都可以抽象成**因果模型**：一个有向无环图（DAG）加上一组结构函数。DAG 的节点是变量，箭头表示父变量对当前变量有输入作用，结构函数说明当前变量怎样由父变量决定。

形式上，一个确定性因果模型可以写成：

$$M = (G, \mathcal{V}, \mathcal{F}), \quad G = (\mathcal{V}, \mathcal{E}), \quad V_i = f_i(\text{Pa}(V_i)).$$

- M ：一个因果模型。
- G ：因果图，也就是有向无环图。
- $\mathcal{V} = \{V_1, \dots, V_n\}$ ：模型中的变量集合。
- $\mathcal{E}$ ：变量之间的有向边集合。
- $\text{Pa}(V_i)$ ：变量 V_i 的父节点集合。
- f_i ：结构函数，规定 V_i 如何由父节点的取值计算出来。

这个定义看似朴素，威力来自它的通用性。神经网络可以视作低层因果模型 M_{low} ：输入 token 进入嵌入层，经过一层层残差更新、注意力计算、MLP 变换，最后产生 logits。每个向量位置、每个模块输出、每个中间激活都可以成为图上的变量。算法也可以视作高层因果模型 M_{high} ：变量是程序中的中间量，箭头是控制流和数据依赖，结构函数是加法、比较、条件分支等操作。

¹¹源视频时间：00:21:42-00:24:54。本图根据该片段字幕与讲解重绘；本地视频在第 4-5 节时间窗无法稳定解码，未伪造成原始截图。

因果抽象的核心翻译

“神经网络实现了某个算法”在这里被翻译成一个可检验命题：存在一个从低层变量到高层变量的对齐关系，使得高层模型和低层模型在对应干预下产生系统一致的输出。这个命题比普通相似性描述更强，因为它要求反事实行为也匹配。

这里的“反事实”指研究者强行设定某个变量的值，观察系统会怎样变化。例如用 Pearl 风格的 $\text{do}(\cdot)$ 表示干预时， $\text{do}(V_i = v)$ 的含义是切断 V_i 原来的父节点输入，把它固定为 v 。在神经网络中，这常对应把某个激活替换成另一个输入运行时得到的激活。

4.3 两位数加法中的 carry-the-one 变量

讲者用两位数加法作为贯穿例子。模型看到 $17 + 25$ 并输出 42。一个自然的高层算法假说来自小学算术：个位 $7 + 5 = 12$ ，于是产生一个二元的进位变量 C ，再把 $C = 1$ 加到十位计算中。这里的 C 就是 carry-the-one 变量，即“向十位进一”的变量。

为了让这个变量变成可测试对象，可以写一个极简高层模型：

$$C = \mathbf{1}[u_1 + u_2 \geq 10], \quad T = t_1 + t_2 + C, \quad Y = 10T + ((u_1 + u_2) \bmod 10).$$

- u_1, u_2 ：两个加数的个位数字，例如 7 和 5。
- t_1, t_2 ：两个加数的十位数字，例如 1 和 2。
- $C \in \{0, 1\}$ ：进位变量；当个位和至少为 10 时取 1。
- T ：输出结果的十位部分。
- Y ：最终两位数输出。
- $\mathbf{1}[\cdot]$ ：指示函数，条件成立时为 1，否则为 0。

在这个模型里， $17 + 25$ 的高层变量取值为 $C = 1$ ，输出 42。另一个输入 $30 + 47$ 的个位 $0 + 7 = 7$ ，所以 $C = 0$ ，输出 77。交换干预的思想是：在运行 $17 + 25$ 的过程中，把它的进位变量替换成 $30 + 47$ 时会出现的值，也就是把 $C = 1$ 替换为 $C = 0$ 。高层模型预测的结果会从 42 变成 32，因为十位少加了一。

这个操作可以写成：

$$Y_{\text{high}}(x_{\text{base}}, \text{do}(C \leftarrow C(x_{\text{src}}))) = 32,$$

其中 $x_{\text{base}} = 17 + 25$ ， $x_{\text{src}} = 30 + 47$ 。

- Y_{high} ：高层加法算法模型的输出。
- x_{base} ：基础输入，即原本会输出 42 的 $17 + 25$ 。
- x_{src} ：来源输入，即提供替换变量值的 $30 + 47$ 。
- $C(x_{\text{src}})$ ：来源输入下的进位变量值，这里为 0。
- $\text{do}(C \leftarrow C(x_{\text{src}}))$ ：把基础运行中的 C 强行替换成来源运行中的 C 。

¹²源视频时间：00:25:03–00:28:29。本图为根据该片段字幕与讲解重绘的教学示意图，未伪造成原始截图。

Two-digit addition: carry-the-one intervention

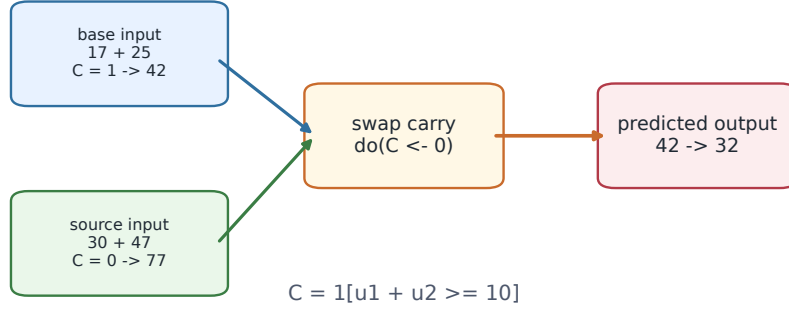


图 12: 两位数加法中的 carry-the-one 变量: 把 $17 + 25$ 的进位 $C = 1$ 替换成 $30 + 47$ 的 $C = 0$, 高层模型预测输出从 42 变为 32。¹²

进位变量区别于输出标签

进位变量 C 是中间计算量, 输出 42 或 32 是下游结果。若研究者只证明某个神经元能预测输出答案, 还没有证明它承担了进位的因果角色。真正的测试要看替换这个变量后, 模型是否按照高层算法预测的方式改变输出。

4.4 结构对应: 同一干预在两个层级上的同构效果

现在把同一个干预搬到神经网络里。假设研究者提出一个低层定位假说: 第 3 层、第二个数字上方的某个神经元或某个激活子空间 S 存储了进位变量 C 。那么可测试的要求是: 在 $17 + 25$ 的前向传播中暂停到这个位置, 把 S 替换成 $30 + 47$ 运行时对应位置的 S , 继续运行模型。如果输出也从 42 变成 32, 这个结果就是结构对应的一条证据。

低层交换干预可以写成:

$$a_{\ell}^{\text{swap}}(x_{\text{base}}, x_{\text{src}}, S) = a_{\ell}(x_{\text{base}}) \odot (1 - m_S) + a_{\ell}(x_{\text{src}}) \odot m_S.$$

- $a_{\ell}(x)$: 输入 x 在第 ℓ 层或某个组件处的激活向量。
- S : 被认为承载高层变量的低层位置、神经元集合或子空间。
- m_S : 选择 S 的掩码; 在 S 上取 1, 其他位置取 0。
- $\odot$: 逐元素乘法。
- a_{ℓ}^{swap} : 把基础输入激活中 S 部分替换成来源输入激活后的混合激活。

若高层和低层真的对齐, 低层干预的输出应当匹配高层干预的输出:

$$\tau_Y(M_{\text{low}}(x_{\text{base}}; \text{do}(S \leftarrow S(x_{\text{src}})))) = Y_{\text{high}}(x_{\text{base}}; \text{do}(C \leftarrow C(x_{\text{src}}))).$$

- M_{low} : 低层因果模型, 即被分析的神经网络。
- M_{high} : 高层因果模型, 即加法算法假说。

- $S(x_{\text{src}})$: 来源输入在低层位置 S 上产生的激活。
- τ_Y : 把低层模型输出映射为高层可比较输出的读出函数，例如把 logits 解码成数字答案。
- 等号右侧: 高层算法在相同交换干预下的预测。

这里的“同构效果”并不要求低层网络长得像高层 DAG。高层 DAG 可以只有几个变量，低层网络可能有数十层和高维残差流。结构对应要求的是干预签名一致：对同一批输入对执行同类交换，两个层级的输出变化系统匹配。若对所有相关输入对都匹配，讲者称之为完美抽象。

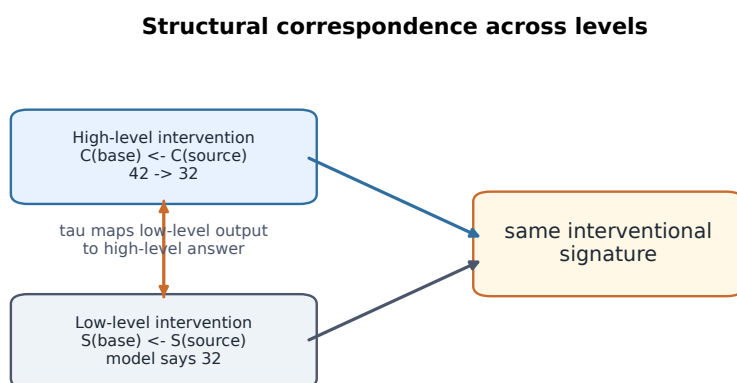


图 13: 结构对应的判据：高层变量 C 的交换干预与低层子空间 S 的交换干预应产生同一种输出签名。¹³

结构对应的判据

一个低层组件承载高层变量，含义是它和该高层变量具有相同的干预性质。相关性、可解码性、局部相似性都只能提供线索；交换干预产生的预测输出才是因果抽象中的硬测试。

4.5 抽象层级与解释深度

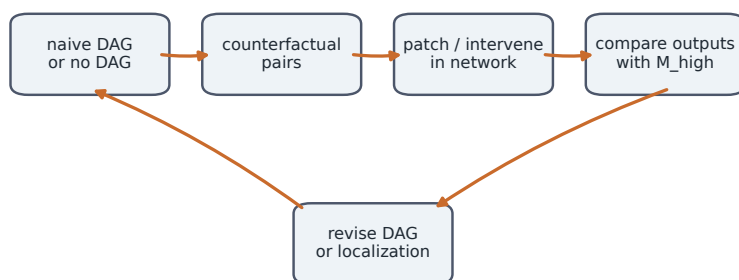
课堂问答补上了一个重要细节：因果抽象提供的是评估框架，科学家仍要提出合适的 DAG。讲者承认，在例子里直接指定“进位变量在第 3 层某个神经元”有偷步成分。现实中，研究者通常从朴素假说或无假说开始，构造输入对，执行补丁实验，观察输出，再修改高层因果模型。这个过程更像科学建模：先画出一个可能有用的地图，再用干预结果检查地图哪里失真。

另一个问答指出：同一个行为可以有多个有效抽象层。进位变量 C 给出很粗的解释，但读者还可以继续追问： $C = 1$ 在向量空间里长什么样？数字是否以 bit 表示？是否还有两个更低层变量先加二、再减一，合起来表现得像进位？这些问题没有被高层抽象消除。因果抽象允许研究者不同粒度上写出不同模型，并分别检验它们。

¹³源视频时间：00:27:07–00:28:45。本图为根据该片段字幕与讲解重绘的教学示意图，未伪造成原始截图。

¹⁴源视频时间：00:29:26–00:31:15。本图为根据问答片段重绘的流程示意图，未伪造成原始截图。

Hypothesis / DAG revision workflow



Causal abstraction evaluates correspondence; scientific judgment proposes and refines it.

图 14: DAG 假说的修订循环：从朴素假说出发，构造反事实输入对，执行补丁或交换干预，再用输出差异修正高层因果模型或低层定位。¹⁴

抽象深度的现实含义

完整理解一台电脑可能需要同时掌握 Python、字节码、机器码和硬件层。神经网络也类似。一个高层因果抽象若被强干预证据支持，已经提供了牵引力；若任务要求更细的机制解释，还可以向下追踪特征几何、子空间结构和跨层信息流。

因此，解释的深度不能只看公式多细。若研究者用另一个同样复杂的神经网络来“抽象”原网络，即使形式上能建立映射，也未必有解释价值。好的高层模型应该比原系统更简单、更可理解，同时保留目标行为的关键反事实结构。因果抽象给出“怎样检验对应”的数学骨架，解释是否令人满意还取决于模型的简洁性、可复用性、可推广性，以及它是否回答了研究者真正关心的问题。

4.6 本章小结

本章把机制可解释性中的“算法解释”改写成因果抽象问题。因果模型由 DAG 和结构函数组成；神经网络可视作低层因果模型，算法可视作高层因果模型。两位数加法中的 carry-the-one 变量提供了最小例子：如果在高层把 $17 + 25$ 的进位替换成 $30 + 47$ 的进位会让 42 变成 32，那么低层中被声称承载进位的组件也应当在对应交换干预下产生同样输出变化。真正的解释价值来自系统性结构对应，单个漂亮例子只能提供局部线索。与此同时，因果抽象只负责检验给定假说；怎样提出足够简洁、足够有解释力的高层 DAG，仍然需要探索实验和科学判断。

5 变量定位：从线性表征到可训练干预

5.1 为什么反事实补丁更贴近模型自然分布

第 4 节留下的技术缺口很具体：如果高层变量 C 存在，研究者怎样在低层网络里找到它？讲者先回应一个关于分布外干预的问题。激活转向常把一个人造构造的向量加到模型内部某处，这个位置可能从未在真实输入中自然出现过。模型仍可能被这种向量操控，但这种操控更像外力刺激，解释的是“怎样让模型做某事”。

交换干预和激活补丁更接近模型的自然分布，因为被替换进去的值来自另一个真实输入的真

实前向传播。它们问的问题也随之改变：系统在正常运行时怎样操纵自己的内部状态，从而完成某个任务？这种干预并不保证绝对安全地留在训练分布上；组合后的混合状态仍可能在全局上罕见。但它至少把局部变量值绑定到真实运行，避免凭空注入任意方向。

反事实补丁的分布直觉

激活补丁把一个真实运行中的内部值搬到另一个真实运行里。它的解释目标是追踪模型自然计算中的可替换部件；激活转向更偏向控制实验，目标是寻找能驱动行为的操控方向。二者都使用干预，科学问题却不同。

On-distribution patching versus steering

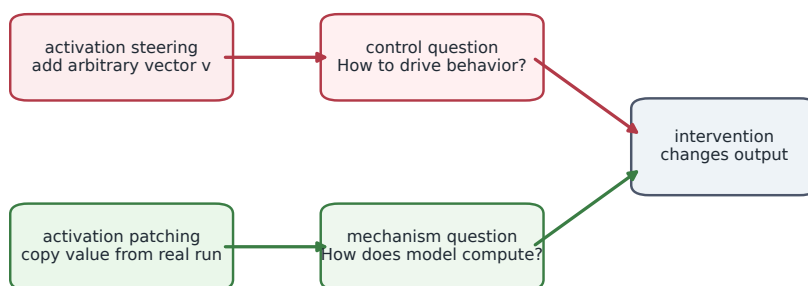


图 15: 反事实补丁与激活转向的科学问题差异：转向更像外部控制向量，补丁把真实运行中的内部值搬到另一个真实运行里，用来追踪自然计算路径。¹⁵

5.2 线性表征假说：变量可能藏在任意方向

若直接替换某个向量坐标，研究者默认高层变量与原始基坐标对齐。以四维激活为例，若只盯着第一个坐标 a_1 ，就是假设它对应进位变量。机制可解释性中的线性表征假说提醒读者：有意义的特征常常位于向量空间中的任意方向，未必恰好落在某个标准坐标轴上。换句话说，变量可能藏在一个方向 $w^\top a$ 里，单个原始坐标看不出来。

一种干净做法是在模型中插入特征化器 f 和逆特征化器 f^{-1} 。模型运行到某个激活 a 后，先把它变换到新的特征坐标 $h = f(a)$ ，再用 $f^{-1}(h)$ 变回原向量空间。如果 $f^{-1}(f(a)) = a$ ，普通前向传播完全不受影响；变化只发生在研究者对 h 的某些坐标做干预时。

最简单的例子是旋转矩阵：

$$h = f(a) = Ra, \quad \tilde{a} = f^{-1}(h) = R^\top h, \quad R^\top R = I.$$

- $a \in \mathbb{R}^d$ ：模型某层某位置的原始激活向量。
- $R \in \mathbb{R}^{d \times d}$ ：正交旋转矩阵，用来把标准坐标轴旋到新的方向。
- h ：旋转后的特征坐标。
- $R^\top$ ：旋转矩阵的转置；在正交条件 $R^\top R = I$ 下，它等于 R 的逆。

¹⁵源视频时间：00:36:20–00:38:10。本图为根据该片段字幕与讲解重绘的教学示意图，未伪造成原始截图。

- $\tilde{a}$: 从特征坐标映射回原空间的向量；无干预时 $\tilde{a} = a$ 。
- I : 单位矩阵。

这个公式的直觉是：旋转不会损失信息，只是换一套坐标尺。若原始坐标像房间里的南北东西，新坐标就像把罗盘转了一个角度后得到的新方向。空间里的点没有变，但某个高层变量可能在新轴上更容易被单独操作。

基坐标与方向特征

基坐标是向量列表里的第 i 个数字；方向特征是某个向量方向上的投影。在线性表征假说下，“变量在哪里”经常意味着“哪个方向或哪个低维子空间承载了变量”，这个问题比“哪个神经元编号等于变量”更贴近向量空间中的实际表示。

Linear representation: features can live in rotated directions

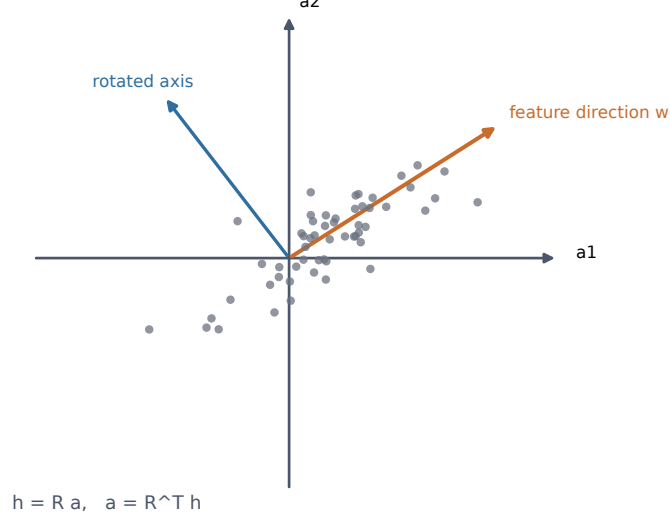


图 16: 线性表征假说的几何直觉：变量可能沿任意方向 w 分布，旋转 $h = Ra$ 只是更换坐标尺，方便在新坐标中单独干预。¹⁶

5.3 分布式交换干预

分布式交换干预（distributed interchange intervention）把第 4 节的交换操作移到特征空间里。流程分为四步：运行基础输入到某层得到 $a_\ell(x_{\text{base}})$ ；运行来源输入得到 $a_\ell(x_{\text{src}})$ ；用特征化器把二者变成 h_{base} 和 h_{src} ；只替换选定特征坐标，再用逆特征化器送回原空间继续前向传播。

公式写成：

$$h_{\text{base}} = f(a_\ell(x_{\text{base}})), \quad h_{\text{src}} = f(a_\ell(x_{\text{src}})),$$

$$h^{\text{swap}} = h_{\text{base}} \odot (1 - m) + h_{\text{src}} \odot m, \quad a_\ell^{\text{DII}} = f^{-1}(h^{\text{swap}}).$$

- f : 特征化器，把原始激活映射到特征坐标。
- f^{-1} : 逆特征化器，把特征坐标映射回模型原始激活空间。
- h_{base} : 基础输入在特征空间中的表示。

¹⁶源视频时间：00:38:10–00:39:52。本图为根据该片段字幕与讲解重绘的几何示意图，未伪造成原始截图。

- h_{src} : 来源输入在特征空间中的表示。
- m : 特征掩码；选择要替换的坐标或子空间。
- a_{ℓ}^{DII} : 分布式交换干预后的激活，将继续送入模型后续层。

若选中的特征坐标真的对应进位变量，那么把 $30 + 47$ 的“无进位”特征搬到 $17 + 25$ 的运行中，应当再次把输出从 42 推到 32。这与第 4 节的高层交换干预完全同构，只是低层变量从“某个显式神经元”推广为“某个旋转后的方向或子空间”。

Distributed interchange intervention

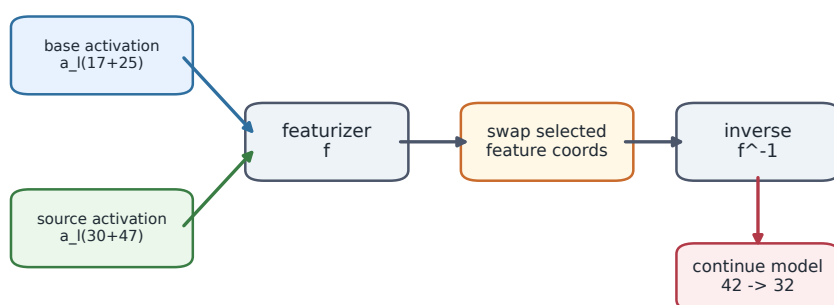


图 17: 分布式交换干预：先用特征化器 f 进入特征空间，替换选定坐标，再经 f^{-1} 回到原激活空间继续前向传播。¹⁷

分布式不等于任意复杂

分布式交换干预允许变量落在方向或子空间里，但仍要求清晰的干预预测。若特征化器太强，它可能学到只适配训练样例的奇异切分。此时结果看起来有效，解释却很脆弱。后续必须用留出样例、简洁约束和跨输入泛化来检查。

当 f 是旋转矩阵时，各个新坐标正交，单独干预一个坐标与干预另一个坐标不会互相纠缠，操作顺序也可交换。若 f 来自稀疏自编码器，编码和解码通常有重构误差，不同特征之间也可能有干扰。实践中常把重构误差加回去，让干预尽量保持可运行，但这需要额外谨慎解释。

5.4 用因果模型监督特征方向

讲者接着回答“怎么找到这个方向”。假设研究者随机初始化一个旋转矩阵 R ，在旋转后的第一维上做进位交换干预。随机方向几乎必然错，模型可能输出 98 或其他无关数字。关键在于，高层因果模型已经告诉研究者期望输出应为 32。这个期望输出可以变成监督信号，用来训练特征化器。

¹⁷源视频时间：00:39:55–00:40:45。本图为根据该片段字幕与讲解重绘的教学示意图，未伪造成原始截图。

训练目标可以概括为：

$$\min_{\theta} \mathbb{E}_{(x_{\text{base}}, x_{\text{src}})} \left[\mathcal{L} \left(Y_{\text{low}}(x_{\text{base}}; \text{DII}_{f_{\theta}, m}(x_{\text{src}})), Y_{\text{high}}(x_{\text{base}}; \text{do}(C \leftarrow C(x_{\text{src}})))) \right) \right],$$

$$\nabla W_{\text{Transformer}} = 0.$$

- $\min_{\theta}$ ：在特征化器参数 θ 上最小化目标函数。
- $\mathbb{E}_{(x_{\text{base}}, x_{\text{src}})}[\cdot]$ ：对基础输入与来源输入组成的交换样本对取平均训练目标。
- θ ：特征化器的可训练参数，例如旋转矩阵的参数。
- $(x_{\text{base}}, x_{\text{src}})$ ：用于交换干预的基础输入和来源输入对。
- $\text{DII}_{f_{\theta}, m}$ ：由特征化器 f_{θ} 和掩码 m 定义的分布式交换干预。
- Y_{low} ：低层神经网络在干预后的输出。
- Y_{high} ：高层因果模型在对应干预后的预测输出。
- $\mathcal{L}$ ：损失函数，例如对目标答案的交叉熵或输出差异损失。
- $W_{\text{Transformer}}$ ：Transformer 的原始权重；梯度为零表示它们被冻结。

这一步非常重要：训练的对象是特征化器或旋转方向，Transformer 本体保持冻结。研究者分析模型，而没有把模型重新训练成另一个系统。监督信号来自高层因果模型的干预行为，因此这类方法可以称为因果变量定位：用“如果这个变量被替换，输出应怎样变化”的结构预测，反过来寻找低层中承载该变量的方向。

Training the featurizer with causal supervision

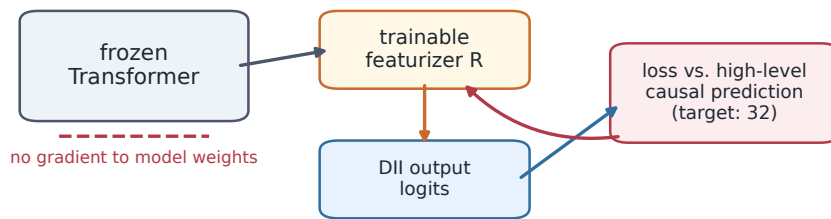


图 18: 用因果模型监督特征方向：Transformer 权重冻结，梯度只进入特征化器或旋转矩阵，目标是让干预输出匹配高层模型预测。¹⁸

¹⁸源视频时间：00:41:00–00:42:50。本图为根据该片段字幕与讲解重绘的训练流程示意图，未伪造成原始截图。

因果监督的关键约束

梯度只流入特征化器或掩码，不能流入被解释的 Transformer 权重。否则实验会从“定位原模型中的变量”滑向“改造模型以满足研究者的变量定义”，解释对象随之改变。

问答中还补充了范围问题。特征化器可以放在一个层、多个层、一个 token 位置或多个 token 位置；若研究者怀疑某个计算跨层发生，可以在多层同时放置特征化器并一起训练。一个高层变量也可以由多个方向承载。简单二元变量有时一个方向足够；更粗厚的对象，例如完整 token 语义，可能需要几十个维度共同承担。

5.5 基于 SAE 的掩码选择

除了训练旋转矩阵，还可以使用已经训练好的稀疏自编码器 (sparse autoencoder, SAE)。SAE 提供一组固定特征：编码器把激活 a 映射为稀疏特征 h ，解码器把 h 还原回激活空间。若研究者不想更新这些特征，可以只学习一个掩码 m ，选择哪些 SAE 特征具有目标干预性质。

可写成：

$$h = E(a), \quad \hat{a} = D(h) + \epsilon, \quad h^{\text{swap}} = h_{\text{base}} \odot (1 - m) + h_{\text{src}} \odot m.$$

- E : SAE 编码器，把激活映射到稀疏特征。
- D : SAE 解码器，把稀疏特征映射回激活空间。
- h : SAE 特征向量，通常维度较高且稀疏。
- h_{base} : 基础输入通过 SAE 编码器得到的稀疏特征。
- h_{src} : 来源输入通过 SAE 编码器得到的稀疏特征。
- $\hat{a}$: SAE 重构出的激活。
- $\epsilon = a - \hat{a}$: 重构误差；实践中常加回以减少损失性编码带来的偏移。
- m : 可训练或可搜索的特征掩码，决定哪些 SAE 特征参与交换。

这种方法的优点是解释对象更接近已有的 SAE 特征词表；缺点是 SAE 特征没有天然的因果变量对齐保证。一个 SAE 特征可能可读、稀疏、常被激活，但对目标输出没有可替换的因果作用。因果掩码训练的目标正是筛掉这类只相关却不担任任务作用的特征。

SAE 掩码与旋转训练的分工

旋转训练在寻找新的连续方向；SAE 掩码在已有特征集合里做选择。前者更灵活，过拟合风险更高；后者更受特征字典限制，但结果往往更容易解释。二者共享同一类因果监督：干预后输出必须匹配高层模型预测。

5.6 和线性探针的关键区别

线性探针也能告诉研究者某个变量在哪里“可被读出”。例如训练一个线性分类器，从每层每个 token 的残差流里预测进位 C 。讲者指出，这类热图往往到处都亮，甚至最后一层的某些位置

¹⁹源视频时间：00:43:07–00:43:56。本图为根据该片段字幕与讲解重绘的教学示意图，未伪造成原始截图。

Autoencoder-based masking over fixed SAE features

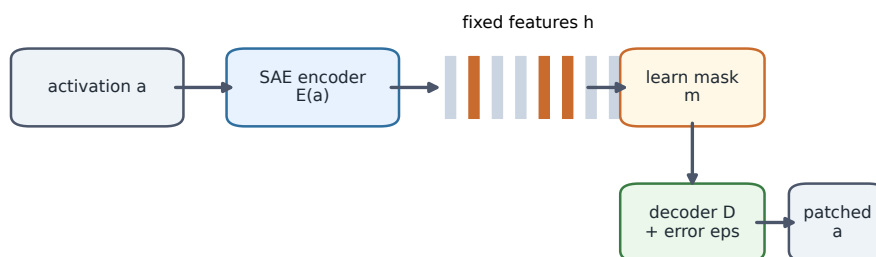


图 19: 基于 SAE 的掩码选择: 保持稀疏自编码器特征固定, 只学习哪些特征参与交换干预, 并用干预输出筛选有因果作用的特征。¹⁹

也能读出相关信息。原因很简单: 神经网络没有义务删除无关信息。一个向量里保留了某个事实, 并不代表后续计算还会使用它。

因果变量定位问的是更强的问题: 找到一些方向, 使得操纵它们会产生与高层因果模型一致的下游变化。探针测量信息的可用性, 交换干预测量信息的使用性。对机制解释而言, 后者更接近“这条路径真的参与了计算”。

线性探针的常见误读

探针成功说明信息在某处可解码, 不能直接推出该处是任务计算的因果路径。尤其在最后层或旁路位置, 可解码信息可能已经没有机会影响目标输出。因果定位要求干预这些方向, 并观察输出是否按高层模型预测改变。

讲者提到, 当这种因果方法有效时, 热图会变得非常局部: 例如进位操作可能集中在最后 token 的中后层若干层中, 探针热图则经常大面积发亮。这种局部性与前面 causal tracing 里“某些 MLP 层负责从 Space Needle 取回 Seattle”的观察相呼应。它不要求所有机制都尖锐集中; 它说明当研究者提出的变量和干预签名足够贴近模型实际计算时, 因果证据可以比相关证据更有辨识度。

问答还给出失败标准。研究者不保证总能找到目标变量。若提出“模型内部应当有这个直观数量”, 训练和干预却无法泛化到留出样例, 合理结论可能是: 这个变量没有以该方式存在, 或高层 DAG 画错了, 或定位空间选错了。能失败正是科学方法的一部分; 否则这套流程只是在给直觉补事后故事。

5.7 本章小结

本章把因果抽象推进到变量定位。激活补丁和交换干预使用真实输入产生的内部值, 因此比任意转向向量更贴近模型自然计算。线性表征假说提醒研究者, 高层变量可能藏在任意方向或子空间中; 特征化器和逆特征化器提供了在新坐标中干预、再映回原空间的工具。分布式交换干预

²⁰源视频时间: 00:48:30–00:50:25。本图为根据该片段字幕与讲解重绘的定性热图示意, 未复现视频或论文中的具体数值。

²¹源视频时间: 00:50:01–00:51:40。本图为根据该片段字幕与讲解重绘的定性热图示意, 未复现视频或论文中的具体数值。

Linear probe heatmap versus causal localization heatmap

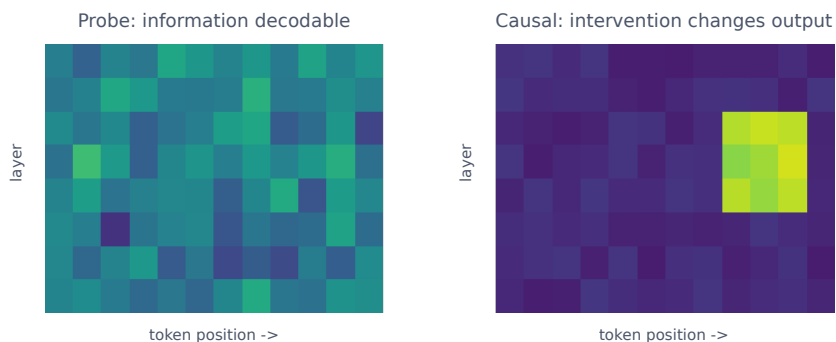


图 20: 线性探针与因果定位的对比: 探针热图显示信息在哪里可解码, 因果定位热图显示干预哪里会按高层模型预测改变输出。²⁰

Localized carry feature: last token, mid-to-late layers



图 21: 局部化的进位因果特征示意: 当定位方法成功时, 进位相关干预效果集中在最后 token 的中后层若干层, 而非整张层位图同时发亮。²¹

保留第 4 节的结构对应要求，只是把“低层变量”从显式神经元推广到旋转方向、SAE 特征或多维子空间。通过冻结 Transformer、训练特征化器或 SAE 掩码，研究者可以把高层因果模型的干预预测变成监督信号。线性探针显示哪里可解码，因果定位检查哪里被模型实际使用；这一区分决定了变量定位能否从相关证据走向机制证据。

6 反事实数据集中的 look-back 机制

6.1 从单个二元变量走向复杂任务

前面几节已经把问题推进到一个更具体的位置：若研究者提出了一个高层因果模型，就需要在神经网络内部找到相应的变量，并通过交换干预检验它是否真的被模型使用。第 6 节从 00:55:20 开始，把这个思想放进一个更接近语言理解的模板任务中。这里不再只追踪一个二元变量，例如加法里的进位；研究对象变成一段小故事，里面有角色、容器、液体状态和提问目标。变量数量上升以后，直觉解释很容易显得顺滑，真正困难的是让每个解释都能承受反事实检验。

讲者把论文中的发现概括为一个 look-back 机制，也可以译作“回看机制”。直观地说，如果模型需要稍后取回上下文中的某个信息，它可以先在该信息附近写下一个小的“位置标记”；等问题出现时，模型复制或重建这个标记，再用它回到上下文中取出真正需要的 token。这个过程像图书馆里的索书号：索书号本身只提供定位信息，却足以把读者带到正确书架。模型内部的位置标记也只承担指向功能，最终答案 token 还需要后续步骤取回。

look-back 机制的核心主张

在 00:55:20–00:58:20 的讲解中，机制被分成三步：先给上下文项目分配位置标记；再从问题里的角色和物体取回这些标记，形成查询；最后用查询找到答案的标记，再通过一次回看取出答案 token。这个主张有因果含义：如果替换的是“指针式位置标记”，输出应当跟随位置变化；如果替换的是“最终答案表示”，输出应当跟随答案 token 本身。

这也是为什么该段材料属于机制可解释性的核心示例。它没有满足于说“模型记住了 Carla 的杯子里是咖啡”，下一步直接把“记住”拆成可干预的内部步骤：标记、查询、解引用。这里的“解引用”来自编程类比：一个变量里存放地址，程序先读地址，再到地址指向的位置读真实内容。对 Transformer 来说，这个地址类比只表示定位功能；实际载体是残差流中可被注意力头和后续层使用的上下文位置信息。

6.2 Bob、Carla、杯子与咖啡：模板任务的结构

讲者在 00:55:51–00:56:17 给出模板故事：Bob fills a bottle with beer; Carla fills a cup with coffee; what does Carla believe the cup contains? 任务的目标答案是 coffee。模板会替换高亮词，例如角色可以从 Bob、Carla 换成其他名字，容器可以从 bottle、cup 换成其他物体，液体可以从 beer、coffee 换成 tea、water 等状态词。

这个模板表面上很简单，实际提供了三类变量：

- 角色变量：例如 Bob、Carla，可记为 character 1、character 2。
- 物体变量：例如 bottle、cup，可记为 object 1、object 2。
- 状态变量：例如 beer、coffee，可记为 state 1、state 2。

²²视频时间：00:55:20–00:58:20。图为根据该时间段字幕与后续清晰幻灯片内容重绘的教学示意图，未伪造成原始截图。

Look-back mechanism: marker -> query -> dereference -> token

Templatic story: Bob / bottle / beer and Carla / cup / coffee

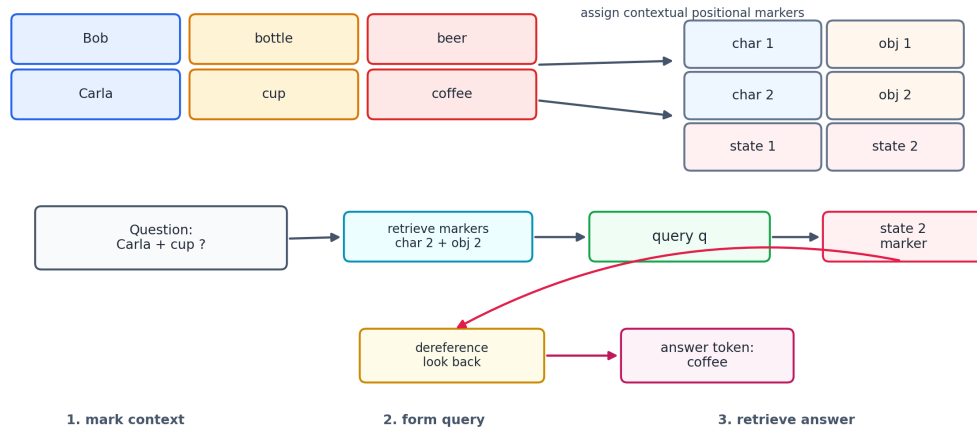


图 22: look-back 机制的三阶段示意：上下文中的 Bob/bottle/beer 与 Carla/cup/coffee 先被整理成上下文的位置标记，问题中的 Carla+cup 取回第二组角色和物体标记形成查询，查询再解引用到 state 2 标记，最终取回 coffee token。此图为忠实机制示意图，用来替代源视频在该窗口无法解码出的 overview 幻灯片。²²

00:56:37–00:57:04 的关键说法是：模型首先把 Bob/bottle/beer 和 Carla/cup/coffee 映射到位置化的角色、物体、状态。Bob 在机制层暂时变成 character 1, bottle 变成 object 1, beer 变成 state 1; Carla、cup、coffee 对应第二组位置。这样做的好处是抽象掉具体词汇，让模型可以处理同一模板下的不同名字、容器和液体。对人类读者来说，这像把故事整理成表格的第 1 行和第 2 行；对模型来说，这些“行号”以内部向量的形式出现在残差流中。

模板任务为什么适合机制定位

模板任务牺牲了开放语言的自然度，换来变量可控性。研究者可以系统交换角色顺序、容器顺序和液体词，从而制造一组“只差在机制假说所关心变量上”的输入对。机制可解释性需要这种可控环境，因为自由文本里的许多差异会同时改变语义、词频、token 边界和位置结构，干预结果很难归因。

这里的“位置”尤其需要谨慎理解。它不同于原始序列中的绝对 token 索引，例如第 17 个 token 或第 23 个 token。后面 01:08:17–01:08:31 的问答明确指出，实验改变了实体出现的绝对 token index，用来排除“模型只读固定 token 位置”的解释。讲者倾向于把它看成更上下文文化、相对化的位置：第一个角色-物体-状态三元组，第二个角色-物体-状态三元组，或者“上文最后一段提到的第三个角色”这类关系。

6.3 三阶段机制：标记、查询、取回答案

00:57:08–00:58:20 的讲解给出了完整计算链。模型看到问题“what does Carla believe the cup contains?”时，需要把 Carla 和 cup 这两个问题词连接到前文中的同一行，然后从该行取回状态词 coffee。讲者描述三阶段可以写成一个紧凑记号，但这个记号只用于帮助读者跟踪变量，不应被误读成论文里的精确神经方程。

先用普通话说清楚：模型为每个相对位置保存一个标记 p_i ，其中 $i = 1, 2$ 表示模板故事里的第几组角色-物体-状态。问题出现后，模型从 Carla 和 cup 的上下文位置取回对应标记，合成一个查询 q 。这个查询再指向同一组里的状态标记，最后通过状态标记取回真正的答案 token。

$$p_i = \text{mark}(c_i, o_i, s_i), \quad q = \text{query}(p_{\text{Carla}}, p_{\text{cup}}), \quad y = \text{lookup}(\text{deref}(q))$$

- i : 模板中的相对项目编号，例如第 1 组 Bob/bottle/beer 与第 2 组 Carla/cup/coffee。
- c_i : 第 i 组的角色 token 或角色变量，例如 Bob、Carla。
- o_i : 第 i 组的物体 token 或物体变量，例如 bottle、cup。
- s_i : 第 i 组的状态 token 或状态变量，例如 beer、coffee。
- p_i : 位置标记，表示第 i 组的上下文化指针信息；它不同于最终答案词。
- p_{Carla} 与 p_{cup} : 从 Carla 和 cup 在前文中的出现处取回的标记。
- q : 由角色标记和物体标记合成的问题表示，可粗略理解为“Carla + cup 应该去查哪一组状态”。
- $\text{mark}(\cdot)$: 为同一组角色、物体和状态生成上下文化位置标记的教学记号。
- $\text{query}(\cdot)$: 把问题中的角色标记和物体标记合成为查询表示的教学记号。
- $\text{deref}(q)$: 解引用步骤，用查询找到对应状态的位置标记。
- $\text{lookup}(\cdot)$: 根据解引用得到的位置取回最终答案 token 的教学记号。
- y : 最终输出 token，在原例中应为 coffee。

这个公式把机制分解为两个层次。第一层是“取到哪一组”：Carla 和 cup 都属于第 2 组，所以查询应该落在第 2 组状态上。第二层是“这一组的状态词是什么”：第 2 组状态词是 coffee。只要把这两个层次分开，后面的反事实实验才有意义；如果所有信息都混在一个最终答案向量里，位置指针和答案 token 就无法被区分。

不要把位置标记误认为位置嵌入

这里的 positional marker 指上下文化的、任务相关的标记，像“这个状态属于第二个角色-容器对”。Transformer 的原始位置嵌入或 rotary position encoding 只提供序列位置信号；它们可以参与形成这种标记，但二者概念边界不同。讲者在 01:08:17–01:08:31 特别强调，相关信息更像相对位置和语境角色，不能简化成绝对 token index。

6.4 为什么要构造精确反事实

00:58:20 以后，讲者缩小到一个具体步骤：如果模型已经有 coffee 的位置标记，并用它取回 coffee token，那么研究者怎样证明自己找到了“位置标记”，同时排除“coffee 的最终答案表示”这个竞争解释？答案是构造一个反事实输入对，让两种假说产生不同输出。

反事实对的设计在 00:58:46–00:59:15 给出：交换 Bob/Carla 与 cup/bottle 的出现顺序，同时重新采样液体，例如把 tea/water 与 beer/coffee 组成两个故事。第一个故事的正确输出是 tea，第二个故事的正确输出是 coffee。若研究者把第一个故事的最终答案表示贴到第二个故事最后 token 的残差流上，模型输出从 coffee 变成 tea 很正常，因为最终答案信息本来就被替换了。

真正的判别实验更精细：把第一个故事中 tea 的位置标记贴到第二个故事中 coffee 的位置标记上。tea 在第一个故事里处于第 1 个状态位置；coffee 在第二个故事里处于第 2 个状态位置。若

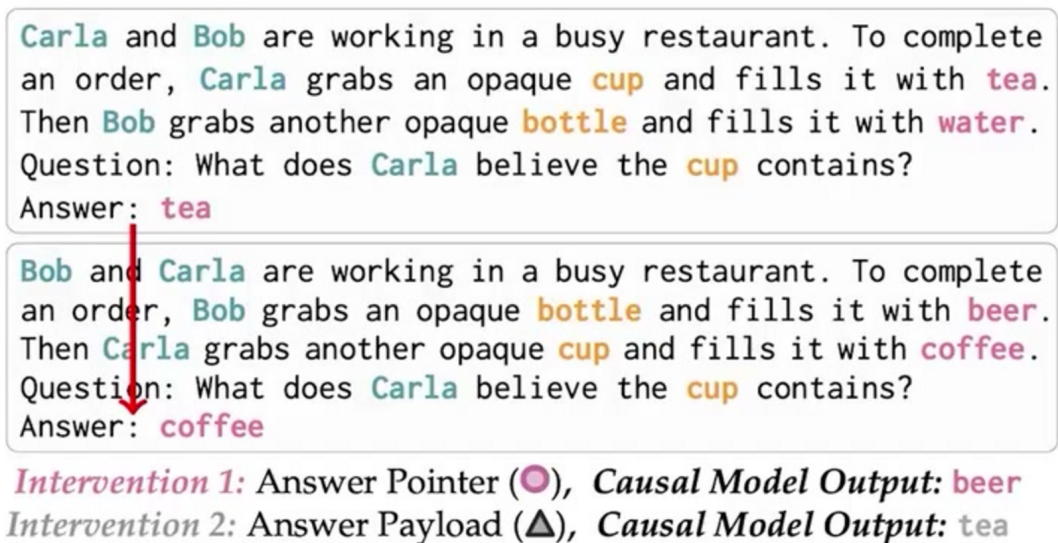


图 23: 反事实故事对的设计: 上方故事中 Carla/cup 对应 tea, 下方原故事中 Carla/cup 对应 coffee; 替换 Answer Pointer 的因果模型输出是 beer, 替换 Answer Payload 的输出是 tea。这个三分输出结构让“位置指针”和“最终答案 token”在实验读数上分开。²³

替换的是最终答案表示, 输出应偏向 tea; 若替换的是位置指针, 第二个故事里的指针会从“取第 2 个状态”变成“取第 1 个状态”, 于是它会在第二个故事内取出第 1 个状态 beer。

反事实设计的判别力

同一个补丁操作如果导致 tea, 说明补丁更像最终答案 token; 如果导致 beer, 说明补丁更像位置指针。beer 在这个实验里很关键, 因为它与原故事的正确答案 coffee、反事实故事的正确答案 tea 都不同; 它对应“把反事实位置指针放进原故事后”得到的目标。

这类设计的价值在于它让输出空间出现三种可区分结果: 原输入答案、反事实答案、指针迁移答案。许多机制解释失败, 原因经常出在测试输入没有把竞争假说拆开, 而非干预工具本身不够复杂。这里的 beer/tea/coffee 三分结构提供了很干净的因果读数。

6.5 位置指针与最终 token 的分离证据

00:59:47–01:01:36 是第 6 节最重要的证据段。讲者说, 如果把 tea 的位置变量覆盖到 coffee 的位置变量上, 就等于把指针从“取第 2 个位置”改成“取第 1 个位置”。在第二个故事里, 第 1 个位置的液体是 beer, 所以观察到 beer 输出, 就是位置标记与最终答案变量分离的证据。

实验结果还显示出层间转换。讲者提到, 在这些例子的最后 token 残差流上做干预, 约 layer 30 开始能看到位置签名, 并且这种签名一路延续到约 layer 50: 输出从 coffee 转向 beer。继续在更后层做同类干预时, 输出会转向 tea, 也就是反事实故事的最终答案。这个现象支持一种阶段性图景: 较早的一段层区间仍在携带“去第几个位置取”的指针信息; 更后面的层逐渐把指针解引用成具体 token 信息。

²³视频时间: 00:58:28–01:00:15。来源帧: 00:59:17, 截图裁剪自课程视频。

层区间证据该怎样读

“layer 30 到 layer 50” 这类表述不应读成硬切分的工程接口。Transformer 层之间的残差流会累积多模块贡献，注意力头和 MLP 模块可能逐步改写同一信号。这里的关键证据是输出类别随干预位置发生结构化变化：中间层更像位置指针，后层更像最终 token。

这也解释了为什么讲者反复强调 counterfactual pair。只有当 coffee、tea、beer 分别承担不同诊断角色时，研究者才能把“替换指针”和“替换答案”区分开。如果反事实设计让指针迁移答案与反事实答案重合，实验会失去判别力；如果词汇、位置和语义同时变化过多，输出变化又会混入词频和上下文偏好。

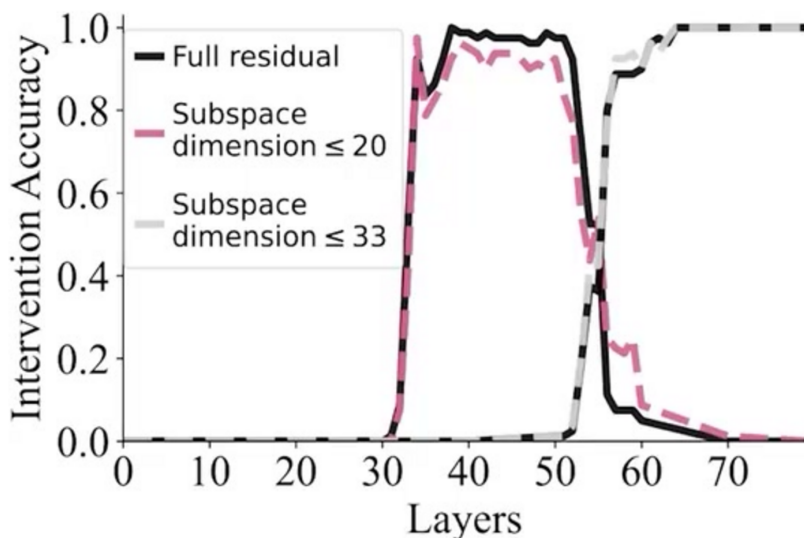


图 24: 位置指针与最终答案表示的分离证据：中间层区间的干预准确率随 Answer Pointer 变化而升高，后层区间则转向 Answer Payload。图中黑线是完整残差流补丁，粉色与灰色虚线显示较小子空间也能复现主要阶段性结构。²⁴

6.6 子空间大小、相对位置与 tokenization 限制

01:02:06–01:03:05 的问答讨论了“子空间维度” (subspace dimension)。讲者解释，图上的实线来自完整向量补丁；另外还做了基于 SVD 分解的 masking。做法大致是收集许多 prompt 下的残差流向量，用 SVD 把它们分解到一组新的方向上，再学习哪些 SVD component 对干预最有用，最后只在这些 component 上执行干预。

这里可以把 SVD 理解成一种数据驱动的坐标旋转。原始残差流可能有上千维，SVD 把观测到的变化方向重新排列，让研究者可以问：“哪些方向最能承载位置标记？”讲者说得很谨慎：他不记得残差流的确切维度，但位置相关信息大约小于 20 维，最终 token 信息大约小于 33 维。这个数字应该写成数量级线索，而不能写成普适常数。

$$a \approx \sum_{k=1}^r \alpha_k u_k, \quad m \odot a_{\text{SVD}} = (m_1 \alpha_1, \dots, m_r \alpha_r)$$

- a : 某个位置、层或组件上的残差流激活向量。

²⁴视频时间：01:00:02–01:01:45。来源帧：01:00:56，截图裁剪自课程视频。

- u_k : SVD 得到的第 k 个方向, 可视为重新组织后的坐标轴。
- α_k : 激活向量在第 k 个 SVD 方向上的系数。
- a_{SVD} : 把激活 a 表示到 SVD 坐标后得到的系数向量, 可理解为 $(\alpha_1, \dots, \alpha_r)$ 。
- r : 保留的 SVD 方向数量。
- m : 学习到的掩码, 决定哪些 SVD component 参与干预。
- $\odot$: 逐坐标相乘, 表示掩码只让部分方向通过。

维度数字要保守使用

“小于 20 维”和“小于 33 维”来自讲者现场回忆与该实验设置, 不能外推为“语言模型中的位置指针只需 20 维”。更稳妥的结论是: 在这个模板任务和被分析模型中, 位置信息似乎可以被压缩到一个相对小的子空间, 且最终 token 表示需要的子空间略大。

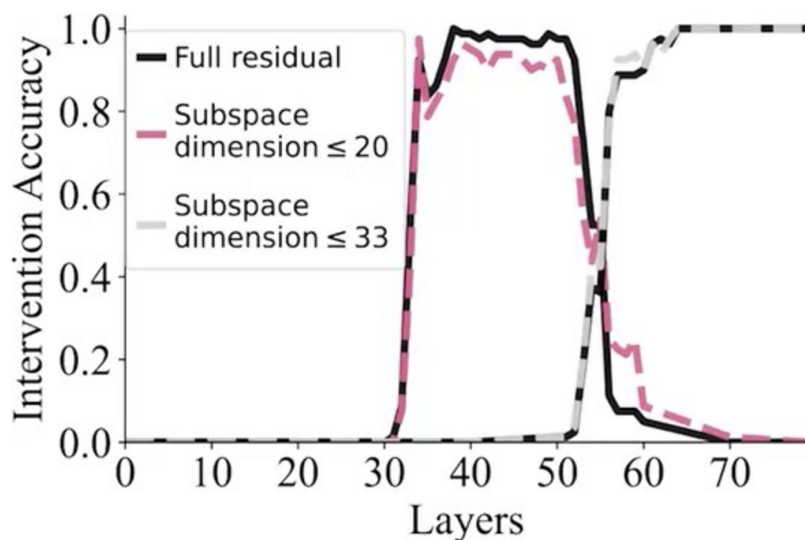


图 25: SVD 掩码与小子空间结果: 实线对应完整向量补丁, 虚线对应只在学习到的 SVD component 子空间中干预; 位置相关信息在图中可由不超过约 20 维的子空间支撑, 最终 token 相关信息对应约 33 维以内的子空间。维度数字只应按本实验语境解释。²⁵

01:03:35–01:05:44 的问答把这个案例推向方法论: 当变量多到九个左右时, 研究者已经很难手工确认每个反事实是否真正控制了混杂因素。讲者因此希望把因果模型表示成 Python object, 用程序检查反事实设计是否对应正确的假说检验, 并进一步用 agent 扩展这种精细分析流程。这里的“扩展”指向可重复、可组合的科学工作流, 核心仍然是小而硬的反事实分析, 不能把机制解释稀释成大规模粗评测。

01:06:49–01:08:55 的问答补充了相对位置问题。只替换角色标记或只替换物体标记, 可能会降低正确答案 logit, 而不一定产生单一清晰替代答案; 当两个查询成分冲突时, 模型可能只是变得不确定。讲者还说, 绝对 token index 的变化实验帮助他们区分了当前位置标记和固定位置嵌

²⁵ 视频时间: 01:02:15–01:03:05。图中结果来自 01:00:56 的清晰截图; 讲者在 01:02:15–01:03:05 对同一结果图解释 SVD 分解、mask 和子空间维度。

入。对语言模型来说，实用查询往往是“上一段第三个角色”这类上下文化关系，固定的“序列第 42 个 token”解释力很弱。

最后，01:09:24–01:10:23 的 strawberry 例子提醒读者：tokenization 会改变任务本身。用户看到的是字符串 strawberry 和字母 r；模型看到的是 tokenizer 切出的 token。若拼写被改到足够离谱，token 边界会改变，模型内部可访问的信息也会改变。一个 token 未必自带字符级分解，除非训练过程让模型学到了这类信息。因此，研究字符计数、拼写变体、词内结构时，必须先问清楚模型实际处理的 token 序列是什么。

6.7 本章小结

第 6 节的核心贡献是把“模型回看上下文取答案”拆成可干预的机制链。Bob/Carla/cup/coffee 模板提供了角色、物体和状态三类变量；look-back 机制提出了位置标记、查询形成和答案解引用三个阶段；反事实对让最终答案补丁与位置指针补丁产生不同输出；beer 这一诊断性输出成为位置标记存在的强证据。

读者应带走三点。第一，位置标记是上下文化指针，不等同于绝对位置嵌入。第二，子空间分析支持“相对少量方向承载特定机制信息”的可能性，但维度数字只能在实验语境内解释。第三，tokenization 是语言模型研究的地基变量；如果研究对象看起来是字符或单词，仍然需要回到 token 级输入确认模型到底看见了什么。

7 总结与延伸：拥抱混合机制

7.1 热图为何看起来平滑或尖锐

01:10:23–01:12:39 的收束讨论从一个听众问题开始：图上的信号有些尖峰，是否意味着某个层突然完成了操作？讲者先承认这类图已经相当干净，但立刻补上更重要的解释：模型里的信息通常不会在单层一次性完成搬运。更常见的情形是，一个 token 的信号在原位置逐层减弱，同时在另一个残差流列上逐层增强，多个注意力头一点点移动信息。

不过，聚合热图（aggregate heatmap）也有陷阱。热图把许多样本对聚合到一起，平滑图案可能来自两种完全不同的生成过程：一种是每个样本内部都逐步移动；另一种是部分样本已经完全移动，部分样本完全停留在原处，平均以后看起来像渐变。二者在热图上相似，机制解释却完全不同。

聚合热图的双重含义

聚合图只能说明“在这批样本上，某些层和位置与输出恢复相关”。它不能单独证明每个样本都经历了同一条平滑计算路径。要区分“样本内渐进移动”和“样本间异质混合”，需要查看单样本恢复、单层恢复或更细粒度的组件干预。

讲者把这个问题接回前文的因果追踪（causal tracing）：如果信息已经完全移动到新位置，恢复旧位置后应当不再有效；如果只有某一层真正执行了移动，那么恢复图会更尖锐。但前面展示的 headline heatmap 往往一次恢复十层窗口，所以视觉上会更“块状”。论文里的 individual restoration 图更渐进，尤其是 MLP 或注意力头分析，很少出现单层“砰”一下完成整个离散操作的情况。

7.2 两项列表中的清晰机制

01:12:47–01:13:14 的最后一个例子把读者带回 Bob/beer、Carla/coffee 这种两项列表。讲者说，在只有两项的设置中，位置机制可以非常干净地解释模型行为。原因很直接：列表短，候选项少，相对位置标记足以把“第 1 项”和“第 2 项”区分开，反事实交换也容易让竞争假说分叉。

这类清晰个案有科学价值。它们像物理实验里的低摩擦轨道：真实世界有风阻和粗糙接触，但先在可控装置里看清楚基本规律，研究者才知道要去复杂场景里找什么。第 6 节的 look-back 分析正是这样的低摩擦装置：模板严格、变量明确、输出可诊断，机制链能被写成位置标记和解引用步骤。

清晰个案的作用

窄任务并不逃避复杂性。它们提供机制母题的可验证样本，例如“先写位置标记，再用查询回看取值”。只要研究者明确边界，这类个案可以成为后续复杂任务分析的参照物。

7.3 长列表中的 U 形退化与词汇直查

01:13:18–01:14:27 给出关键转折：当列表从两项扩展到更长列表时，同一个位置机制在中间位置退化。讲者提到图上出现 U 形结果：开头位置表现很好，中间位置性能明显下降，末尾位置又升高。这个模式说明，模型在长列表中不总是依靠同一种位置指针。

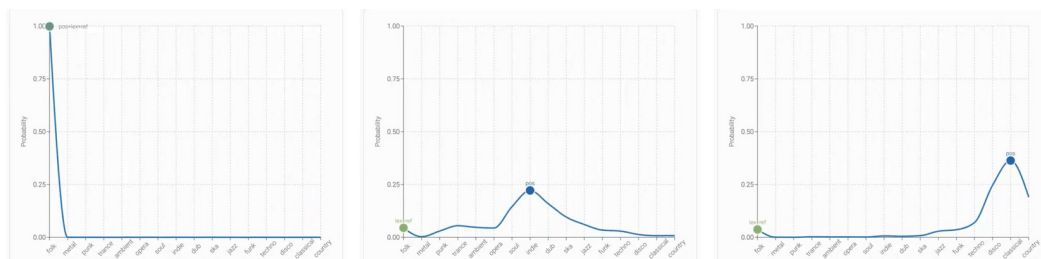


图 26: 长列表中的位置机制退化：三帧从左到右展示列表开头、中间、末尾附近的读数。开头位置的 positional 读数接近满分，中间位置显著下降，靠近末尾时又回升；这正是讲者用来说明长列表里会混入直接词汇查找等机制的 U 形证据。²⁶

在中间位置，模型会更多使用 direct lexical lookup，即直接词汇查找。例子是 Pete loves jam：位置机制会说“Pete-jam 这一对在第 3 个位置，所以查询时使用 position 3 marker 去取 jam”；直接词汇查找会绕过位置号，直接用 Pete 这个 token 或其表示去检索 jam。两种机制都能在某些输入上产生正确答案，但它们对反事实干预的响应会不同。替换位置标记时，位置机制应当跟着位置走；替换名字或词汇关联时，词汇直查会更敏感。

这个 U 形现象也很符合语言模型的训练压力。列表开头和结尾通常有更突出的序列边界信号，容易被位置或 recency 线索抓住；中间项缺少边界优势，模型可能转向更局部、更词汇化的关联。这里不能把 U 形解释成唯一机制，但它至少提醒研究者：随着任务规模增长，原先干净的算法会和启发式策略叠在一起。

7.4 从清晰个案到混合机制的研究姿态

01:14:00–01:14:55 是讲者实质性的 closing synthesis。两项列表带来“非常锐利、非常干净”的分析；稍微复杂一点的设置就会出现多机制混合。具体输入类型、问题类型和列表位置都会调

²⁶视频时间：01:13:20–01:14:55。来源帧：01:14:05、01:14:10、01:14:15，截图组合自课程视频。

节语言模型实际采用的机制。模型可以同时学到层叠的算法和启发式规则，而这些算法与规则来自训练过程中的统计压力。

这段收束很重要，因为它防止读者把第 6 节的漂亮案例误读成普遍宣言。机制可解释性真正需要的姿态有两层。第一层是追求窄场景中的清晰因果证据：构造反事实、定位变量、让竞争假说在输出上分叉。第二层是接受复杂场景中的混合结构：同一个任务族里，模型可能在短列表使用位置标记，在长列表中段转向词汇直查，在某些边界项又回到位置线索。

算法母题与混合机制

算法母题指反复出现的计算模式，例如“标记-查询-解引用”。混合机制指多个母题或启发式在同一任务族内共同发挥作用。前者帮助研究者命名可复用结构，后者提醒研究者检查适用边界。好的机制解释应同时报告“在哪些输入上成立”和“在哪些输入上被别的策略替代或补充”。

从研究流程看，这节课最后落在一个朴素但严格的要求上：不要只画一张漂亮热图，也不要只讲一个顺滑故事。需要把故事写成变量和干预；需要让反事实输入区分竞争解释；需要在更长、更乱、更接近真实语言的场景中复查边界。机制可解释性的科学性，就来自这种从清晰个案出发、再主动追踪混合与失效的循环。

7.5 本章小结

第 7 节把整堂课的结尾压缩成一个研究姿态：清晰机制值得追求，但聚合图、长列表和真实语言都会暴露混合结构。热图的平滑或尖锐需要细粒度恢复实验来解释；两项列表里的位置机制可以非常干净；长列表中间位置出现 U 形退化，并引入直接词汇查找；最终的可解释性工作应当同时保留窄任务的因果锋利度和复杂任务的机制多样性。

这也是第 6 节反事实案例的延伸意义。look-back 机制给出了一个可检验的算法母题；closing discussion 提醒读者，算法母题一旦进入更丰富的输入分布，就可能与其他启发式共同工作。机制解释的目标在于用合适粒度画出哪些变量、哪些路径、哪些输入条件共同决定了输出，避免把模型压扁成单一故事。

7.6 全课方法速查

把整堂课压成操作流程，可以得到一条从粗到细的检查链。第一步是定义行为与目标读数，例如 logit、准确率或反事实输出类别。第二步是提出高层变量，例如进位 C 、位置标记 P 、答案载荷 A 。第三步是设计能让竞争解释分叉的输入对，让 Y_{high} 对不同假说给出不同预测。第四步才进入低层定位：残差流、注意力头、MLP、方向、SAE 特征或 SVD 子空间，都只是候选承载位置。

读图时的四个问题

看到热图或补丁曲线时，读者可以按四个问题审稿：干预变量是否清楚；来源输入与基础输入是否只改变必要变量；输出读数是否能区分竞争假说；聚合结果是否有单样本或更细粒度组件实验支撑。若这些问题答不上来，图像只能算线索，尚未构成强机制证据。

因此，本讲所有方法都围绕同一件事旋转：先写清变量 Z 与输出 Y 的反事实承诺，再检查某个低层位置、方向或子空间 S 的替换效果是否匹配这个承诺。激活转向、因果追踪、因果抽象、分布式交换干预和 look-back 分析的差别，主要体现在 S 的选择方式和证据强度上。